

DAT — Data a kódování

Maturitní zápisky · 25 otázek · příprava 30 min, obhajoba 15 min · teorie + praktická simulace

Obsah

1. HTML5 a sémantika
2. Bootstrap a návrh designu
3. Flexbox a rozmístění prvků
4. CSS Grid a tvorba layoutu
5. Pozicování prvků a z-index
6. CSS animace a transformace
7. Tabulky v HTML a stylování
8. Datové typy a pole
9. Spojové struktury a stromy
10. Podprogramy a lambda funkce
11. Kolekce: pole, stack, queue, dictionary
12. Souborový systém a streamy
13. Paralelní a asynchronní programování
14. Verzovací systémy: Git a GitHub
15. ER model a návrh databáze
16. SQL — výběr a filtrování dat
17. REST API v ASP.NET
18. Razor Pages — zpracování požadavku
19. ASP.NET Tag Helpers a formuláře
20. Next.js a SSR vs. CSR
21. ORM
22. React — komponenty a props
23. React — hooks
24. React Router
25. Správa stavů aplikace v Reactu

1 HTML5 a sémantika

Elevator pitch

HTML5 je aktuální verze značkovacího jazyka pro tvorbu webových stránek. Sémantika znamená, že místo bezvýznamových `<div>` používáme tagy nesoucí význam (`<header>` , `<article>`) — kód je tak srozumitelnější pro vyhledávače, čtečky obrazovky i kolegy.

TEORETICKÝ ZÁKLAD

- **Doctype:** `<!DOCTYPE html>` — povinné na 1. řádku, přepne prohlížeč do standardního režimu.
- **Kostra dokumentu:** `<html>` → `<head>` (metadata) + `<body>` (obsah).
- **Povinné meta tagy:** `charset="UTF-8"` , `viewport` (responzivita), `title` , `description` .
- **Sémantické tagy HTML5:** `<header>` , `<nav>` , `<main>` (jen 1×), `<section>` , `<article>` , `<aside>` , `<footer>` , `<figure>` , `<time>` , `<mark>` .
- **Hierarchie nadpisů:** právě jeden `<h1>` na stránce, nepřeskakovat (h2 → h4 ❌).
- **Typografie:** `<p>` , `<strong>` (důraz/význam), `<em>` (zvýraznění), `<b>/<i>` (jen vizuální), `<br>` , `<hr>` , `<blockquote>` , `<cite>` , `<code>` , `<kbd>` .
- **Multimédia:** `<img alt>` , `<picture>` + `<source>` (responzivní obr.), `<video controls>` , `<audio>` .
- **Globální atributy:** `id` , `class` , `data-*` , `lang` , `title` , `aria-*` .
- **Přístupnost (a11y):** popisky alt, label u inputů, kontrast, čitelná struktura, ARIA jen pokud nemá HTML nativně.
- **Validace** přes W3C validator — odhalí špatné zanoření, chybějící atributy.

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: sémantické tagy HTML5, struktura stránky, typografie, hierarchie nadpisů, validace, přístupnost.

Zadání praktického úkolu

Vytvořte staticky sémanticky korektní stránku „*Životopis programátora*“ v souboru `index.html` .

1. Sestavte kostru dokumentu s českou lokalizací, kódováním UTF-8 a viewportem.
2. V `<header>` uveďte jméno (`h1`), profesi a navigaci se třemi odkazy (O mně, Zkušenosti, Kontakt) v `<nav>` .
3. V `<main>` připravte tři `<section>` odpovídající navigaci. Každou sekci uveďte vlastním `<h2>` .
4. V sekci „Zkušenosti“ použijte `<article>` pro každou pozici. Datum vyznačte tagem `<time datetime="...">` , technologie zvýrazněte `<mark>` .
5. Doplněte `<aside>` s krátkým citátem (`<blockquote>` + `<cite>`).
6. Stránku zakončete `<footer>` s rokem a kontaktem.
7. Stránka musí projít W3C validátorem.

? Otázky k obhajobě

- Vysvětlíte rozdíl mezi `<section>` a `<article>` .
- Proč nepoužíváme `<b>` , ale `<strong>` pro důraz?
- K čemu slouží `alt` atribut a co se stane, když chybí?

KUCHARKA — POSTUP NA POTÍTKU

1. Začnu prázdným souborem `index.html` a šablonou `! + Tab` v editoru (Emmet).
2. Doplním `lang="cs"` a `<title>` .
3. Skicuji strukturu na papíře: header → nav → main(3 sekce) → aside → footer.
4. Píšu od shora dolů, každou sekci uzavírám hned po otevření (zabrání nezavřeným tagům).
5. U každého obrázku doplním `alt` , u `<time>` ISO datum.
6. Nakonec validuji v devtools (Lighthouse → Accessibility, validator.w3.org).

KLÍČOVÉ ÚRYVKY KÓDU

 `index.html` — kostra a sémantická struktura

```
<!DOCTYPE html>
<html lang="cs">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Životopis programátora Antonína R.">
  <title>Antonín Reiser – programátor</title>
</head>
<body>
  <header>
    <h1>Antonín Reiser</h1>
    <p>Junior fullstack vývojář</p>
    <nav aria-label="hlavní">
      <ul>
        <li><a href="#about">O mně</a></li>
        <li><a href="#xp">Zkušenosti</a></li>
        <li><a href="#contact">Kontakt</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="about">
      <h2>O mně</h2>
      <p>Studuji 4. ročník SPŠE...</p>
    </section>

    <section id="xp">
      <h2>Zkušenosti</h2>
      <article>
        <h3>Brigáda – webový vývoj</h3>
        <p><time datetime="2024-07">Léto 2024</time> – práce s
          <mark>React</mark> a <mark>ASP.NET</mark>.</p>
      </article>
    </section>

    <aside>
      <blockquote>Programátor je překladatel mezi člověkem a strojem.
        <cite>– neznámý autor</cite>
      </blockquote>
    </aside>
  </main>

  <footer>
    <p>&copy; 2026 Antonín Reiser</p>
  </footer>
```

```
</body>  
</html>
```

2 Bootstrap a návrh designu

Elevator pitch

Bootstrap je nejpopulárnější CSS framework — sada hotových tříd a komponent, které urychlují tvorbu responzivních a vizuálně konzistentních webů. Místo psaní vlastního CSS přidáváme připravené třídy.

TEORETICKÝ ZÁKLAD

- **Co Bootstrap přináší:** reset (Reboot), 12-sloupcový grid, flex utility, hotové komponenty (navbar, modal, card, form), JS plugíny.
- **Instalace:** CDN (`<link>` + `<script>`) nebo npm (`npm i bootstrap`) + Sass varianta pro úpravy proměnných.
- **Container:** `.container` (fixní šířky podle breakpointu), `.container-fluid` (vždy 100 %).
- **Grid:** `.row` obsahuje sloupce `.col` , `.col-6` , `.col-md-4` , `.col-lg-3` ... součet ve `.row` ≤ 12.
- **Breakpointy:** `xs` <576, `sm` ≥576, `md` ≥768, `lg` ≥992, `xl` ≥1200, `xxl` ≥1400.
- **Spacing utility:** `m-3` margin, `p-2` padding, varianty `mt-/mb-/ms-/me-/mx-/my-` ; hodnoty 0–5 (i auto).
- **Flex utility:** `d-flex` , `justify-content-*` , `align-items-*` , `flex-wrap` , `gap-*` .
- **Komponenty:** Navbar, Card, Modal, Carousel, Accordion, Toast, Alert, Form, Button.
- **Customizace:** CSS proměnné (`--bs-primary`) nebo přepsání Sass proměnných před importem.
- **Bootstrap Icons** — vlastní fontová sada ikon přes `<i class="bi bi-check">` .

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: Bootstrap 5, grid systém, komponenty (navbar, card), responzivita, breakpointy, utility třídy.

Zadání praktického úkolu

Vytvořte responzivní landing page pro fiktivní kavárnu „Café Maturita“ v souboru `index.html`. Použijte Bootstrap 5 z CDN.

1. Stránka musí obsahovat **responzivní navbar** s logem (text), 3 odkazy a tlačítkem „Rezervovat“, které se na malých displejích sbalí do hamburger menu.
2. Pod navigací **hero sekci** s nadpisem, popisem a tlačítkem (využijte třídy `.bg-primary`, `.text-white`, `.py-5`).
3. Sekce **Naše nabídka**: 6 produktů zobrazených v gridu jako `card`. Layout: na `md` 3 sloupce, na `sm` 2, na `xs` 1.
4. Footer s adresou a sociálními ikonami (Bootstrap Icons).
5. Žádné vlastní CSS — vše řešte utility třídami.

? Otázky k obhajobě

- Jak funguje 12-sloupcový grid a co znamená `col-md-4`?
- Co je *mobile-first* a jak se promítá do Bootstrap tříd?
- Rozdíl mezi `.container` a `.container-fluid`?

KUCHARKA — POSTUP

1. Vložím CDN `<link>` Bootstrap CSS do `<head>` a `<script>` bundle před `</body>`.
2. Zkopíruji navbar z dokumentace (Components → Navbar) a upravím odkazy.
3. Hero: `<section class="bg-primary text-white py-5 text-center">`.
4. Karty: `.row.g-4 > .col-12.col-sm-6.col-md-4 > .card`.
5. Footer: `.bg-dark.text-light.py-4`, ikony Bootstrap Icons.
6. Test: zmenšuji okno a sleduji breakpointy (devtools responsive view).

KLÍČOVÉ ÚRYVKY KÓDU

 CDN připojení (head + body)


```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-i
<!-- před </body>: -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bund
```

☰ Responzivní navbar

```
<nav class="navbar navbar-expand-md navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">Café Maturita</a>
    <button class="navbar-toggler" type="button"
      data-bs-toggle="collapse" data-bs-target="#nav">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="nav">
      <ul class="navbar-nav me-auto">
        <li class="nav-item"><a class="nav-link" href="#menu">Nabídka</a></li>
        <li class="nav-item"><a class="nav-link" href="#about">O nás</a></li>
        <li class="nav-item"><a class="nav-link" href="#contact">Kontakt</a></li>
      </ul>
      <a href="#" class="btn btn-warning">Rezervovat</a>
    </div>
  </div>
</nav>
```

☰ Grid karet — 1/2/3 sloupce podle breakpointu

```
<section id="menu" class="py-5">
  <div class="container">
    <h2 class="mb-4">Naše nabídka</h2>
    <div class="row g-4">
      <div class="col-12 col-sm-6 col-md-4">
        <div class="card h-100">
          
          <div class="card-body">
            <h5 class="card-title">Espresso</h5>
            <p class="card-text">Klasické italské espresso.</p>
            <span class="fw-bold text-primary">55 Kč</span>
          </div>
        </div>
      </div>
      <div>
        <!-- ... další karty ... -->
      </div>
    </div>
  </div>
</section>
```

3 Flexbox a rozmístění prvků

Elevator pitch

Flexbox je 1D layout systém pro rozmístění prvků v řadě nebo sloupci. Řeší dynamické zarovnání, mezery a velikosti — bez floatů a triků s `vertical-align`.

TEORETICKÝ ZÁKLAD

- **Aktivace:** rodiči nastavíme `display: flex` (nebo `inline-flex`) → stane se z něj *flex container*, jeho přímí potomci jsou *flex items*.
- **Hlavní vs. příčná osa:** hlavní osa je směr, ve kterém se řadí prvky (default `row` = horizontálně). Příčná je kolmá.
- **Vlastnosti containeru:**
 - `flex-direction` : `row` | `row-reverse` | `column` | `column-reverse`.
 - `flex-wrap` : `nowrap` (default) | `wrap` — povolí zalomení do víc řádků.
 - `justify-content` (hlavní osa): `flex-start` | `center` | `flex-end` | `space-between` | `space-around` | `space-evenly`.
 - `align-items` (příčná): `stretch` (default) | `center` | `flex-start` | `flex-end` | `baseline`.
 - `align-content` — zarovnání více řádků (jen při `wrap`).
 - `gap` , `row-gap` , `column-gap` — mezery mezi prvky (čisté řešení).
- **Vlastnosti itemů:**
 - `flex-grow` — kolik volného místa si prvek vezme (poměr).
 - `flex-shrink` — jak se smrští, když je málo místa.
 - `flex-basis` — výchozí velikost před rozdělením.
 - Zkratka `flex: 1 1 200px;` = `grow shrink basis`.
 - `order` — vizuální pořadí (default 0).
 - `align-self` — překryje `align-items` jen pro tento prvek.
- **Použití:** nabary, formulářové řádky, vystředění (jeden item + `justify/align center`), karetní pruhy.
- **Limity:** Flexbox je 1D — pro 2D layout (řádky × sloupce) je lepší CSS Grid.

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: flex container, justify-content, align-items, gap, flex-grow, responzivní wrap, vertikální zarovnání.

Zadání praktického úkolu

Vytvořte stránku `index.html` + `style.css` s rozhraním pro mediální dashboard, ve kterém využijete **výhradně Flexbox** (žádné floats, žádný Grid).

1. **Header (top bar):** vlevo logo, uprostřed seznam odkazů, vpravo avatar uživatele. Vše vertikálně zarovnané, mezery 16 px.
2. **Hero blok:** nadpis a tlačítko vystředěné horizontálně i vertikálně, výška 80vh.
3. **Karetní lišta** se 4 kartami (každá obsahuje obrázek, nadpis, popis). Karty mají stejnou šířku (`flex: 1`), na šířce pod 768 px se zalomí a roztáhnou na celou šířku.
4. **Footer:** vlevo copyright, vpravo 3 ikony — využijte `justify-content: space-between` .

? Otázky k obhajobě

- Co je hlavní a příčná osa? Co se stane, když změním `flex-direction` ?
- Co dělá `flex: 1` a jak souvisí s `flex-grow` ?
- Jak vystředím prvek v rodiči pomocí Flexboxu?

KUCHARKA — POSTUP

1. Připravím sémantický HTML kostru (header/main/footer).
2. Reset: `*{box-sizing:border-box} body{margin:0;font-family:sans-serif}` .
3. Header: `display:flex; align-items:center; justify-content:space-between; gap:1rem;` .
4. Hero: `display:flex; flex-direction:column; justify-content:center; align-items:center; height:80vh;` .
5. Karty: rodič `display:flex; flex-wrap:wrap; gap:1rem;` ; dítě `flex:1 1 240px` (grow + min šířka).
6. Media query `max-width:768px` → karty `flex-basis:100%` .
7. Otestuji v devtools v různých šířkách.

KLÍČOVÉ ÚRYVKY KÓDU

 `style.css` — flex header + karty

```

/* Top bar */
.header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  padding: 1rem 2rem;
  background: #0f766e;
  color: #fff;
}

.header nav ul {
  display: flex;
  gap: 1.25rem;
  list-style: none;
  margin: 0; padding: 0;
}

/* Hero – vystředění */
.hero {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  height: 80vh;
  text-align: center;
  gap: 1rem;
}

/* Karetní lišta */
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
  padding: 2rem;
}

.card {
  flex: 1 1 240px;          /* grow shrink basis */
  background: #fff;
  border-radius: 12px;
  padding: 1rem;
  box-shadow: 0 4px 12px rgba(0,0,0,.1);
}

/* Mobil */
@media (max-width: 768px) {
  .card { flex-basis: 100%; }
}

```

```
.header { flex-direction: column; }  
}
```

4 CSS Grid a tvorba layoutu

Elevator pitch

CSS Grid je 2D layout systém — současně spravuje řádky i sloupce. Hodí se na celostránkové layouty, dashboardy a galerie, kde Flexbox naráží na limity.

TEORETICKÝ ZÁKLAD

- **Aktivace:** `display: grid` na rodiči.
- **Definice mřížky:**
 - `grid-template-columns: 1fr 2fr 1fr;` — 3 sloupce v poměru 1:2:1.
 - `grid-template-rows: 80px auto;`
 - **Jednotka `fr`** rozdělí volný prostor poměrně.
 - `repeat(12, 1fr)` — opakování; `minmax(200px, 1fr)` — minimum a maximum.
 - `auto-fit` / `auto-fill` + `minmax` → automatický responzivní grid.
- **Mezery:** `gap: 1rem;` (resp. `row-gap` , `column-gap`).
- **Umísťování prvků:**
 - `grid-column: 1 / 3;` (od linky 1 do linky 3) nebo `span 2` .
 - `grid-row: 1 / span 2;`
- **Pojmenované oblasti** (template-areas): nejčitelnější způsob layoutu —

```
grid-template-areas:  
  "header header header"  
  "nav    main    aside"  
  "footer footer footer";
```

Prvkům se přiřadí `grid-area: header;` .

- **Zarovnání:** `justify-items` / `align-items` uvnitř buňky, `justify-content` / `align-content` celé mřížky, `place-items: center` = oboje najednou.
- **Implicitní řádky:** `grid-auto-rows` , `grid-auto-flow: row|column|dense` .
- **Grid vs. Flexbox:** Grid pro 2D rozvržení (layouty, mřížky), Flex pro 1D (komponenty, řádky).

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: grid-template-columns/rows, grid-template-areas, fr, gap, repeat, auto-fit + minmax, responzivní layout.

Zadání praktického úkolu

Sestavte layout magazínu „TechBlog“ v souborech `index.html` a `style.css`, kde základ vzhledu tvoří výhradně CSS Grid.

1. Stránka má 4 oblasti uspořádané pomocí `grid-template-areas`: *header* (přes celou šířku), *nav* (úzký levý sloupec), *main* (široký prostřední), *aside* (pravý sloupec) a *footer* (přes celou šířku).
2. V sekci `main` vytvořte galerii kartiček s články. Použijte `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));` — automaticky se přizpůsobí šířce.
3. Jeden článek označte jako „doporučený“ — má zabírat 2 sloupce a 2 řádky (`grid-column: span 2; grid-row: span 2;`).
4. Pod 768 px změňte layout na jednosloupcový (oblasti pod sebou).

? Otázky k obhajobě

- Rozdíl mezi `fr` a `%`?
- Co dělá `auto-fit` oproti `auto-fill`?
- Kdy zvolit Grid a kdy Flexbox?

KUCHARKA — POSTUP

1. Sémantický HTML kostru — `header`, `nav`, `main`, `aside`, `footer` jako přímí potomci jednoho `.layout`.
2. Na `.layout` nastavím `display:grid` + `grid-template-columns` + `grid-template-areas`.
3. Každé sekci přiřadím `grid-area: header` atd.
4. V `main` vnoříme druhý grid (galerie) s `repeat(auto-fit, minmax(220px,1fr))`.
5. Doporučenému článku přidám třídu `.featured` s `grid-column: span 2; grid-row: span 2;`.
6. Media query přepne areas na jeden sloupec.

KLÍČOVÉ ÚRYVKY KÓDU

 `style.css` — celostránkový grid


```

.layout {
  display: grid;
  min-height: 100vh;
  grid-template-columns: 200px 1fr 240px;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header header"
    "nav    main    aside"
    "footer footer footer";
  gap: 1rem;
}

header { grid-area: header; }
nav    { grid-area: nav;   }
main   { grid-area: main;  }
aside  { grid-area: aside; }
footer { grid-area: footer; }

/* Responzivní galerie článků uvnitř main */
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}

.gallery .featured {
  grid-column: span 2;
  grid-row: span 2;
}

@media (max-width: 768px) {
  .layout {
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "nav"
      "main"
      "aside"
      "footer";
  }

  .gallery .featured { grid-column: span 1; grid-row: span 1; }
}

```

5 Pozicování prvků a z-index

Elevator pitch

CSS pozicování ovládá, kam se prvek vykreslí mimo standardní tok dokumentu.

Vlastnost `position` nabízí 5 hodnot, `z-index` řídí pořadí prvků na ose Z.

TEORETICKÝ ZÁKLAD

- **Hodnoty `position` :**
 - `static` — výchozí, prvek je v normálním toku, `top/left` nemají vliv.
 - `relative` — zůstává v toku, ale lze ho posunout přes `top/right/bottom/left` ; vytváří kotvu pro absolutní potomky.
 - `absolute` — vyjmut z toku, polohuje se podle nejbližšího pozicovaného předka (jinak `<html>`).
 - `fixed` — vyjmut z toku, kotví se k viewportu (zůstává při scrollu).
 - `sticky` — kombinace: chová se relativně, dokud nedoroluješ na limit, pak „přilepí“ k okraji.
- **Stacking context** — prostředí, v němž `z-index` srovnává prvky. Vytvoří se mj. u `position ≠ static + z-index ≠ auto`, `opacity < 1` , `transform` , `filter` , `will-change` , `isolation: isolate` .
- **z-index** funguje jen v rámci stejného stacking contextu — vyšší hodnota = výš. Hodnota mimo kontext nevykouká ven.
- **Obtékání:** `float: left/right` (legacy způsob obtékání textu kolem obrázku); `clear` ukončí obtékání. Layouty dnes řeší Flex/Grid.
- **Praktická pravidla:**
 - Modální okno: `position:fixed; inset:0; + overlay s z-index` .
 - Tooltip: `position:absolute` v relativním rodiči.
 - Sticky header: `position:sticky; top:0; .`
 - Vystředění: `position:absolute; inset:0; margin:auto; nebo top:50%; left:50%; transform:translate(-50%,-50%); .`

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: position (relative/absolute/fixed/sticky), z-index, stacking context, modální okno, sticky header.

Zadání praktického úkolu

Vytvořte stránku `index.html` s těmito interaktivními pozicovanými prvky:

1. **Sticky header** nahoře, který při scrollu zůstává viditelný.
2. **Plovoucí tlačítko** „nahoru“ v pravém dolním rohu — fixed, vždy viditelné.
3. **Galerie obrázků**; po kliknutí na obrázek se otevře **modální okno** přes celou obrazovku s polopruhledným pozadím a centrálně zobrazeným obrázkem. Zavírá se kliknutím na X (vpravo nahoře) nebo na pozadí.
4. Na každém obrázku v galerii má být v levém horním rohu **štítek „NOVÉ“** (absolutní pozicování v relativním rodiči).
5. Zajistěte správné pořadí přes `z-index` — modální okno nad vše ostatní.

? Otázky k obhajobě

- Rozdíl mezi `absolute` a `fixed` ?
- Co je stacking context a kdy vzniká?
- Proč může `z-index: 9999` nezabrat?

KUCHARKA — POSTUP

1. Header: `position: sticky; top: 0; z-index: 10; .`
2. Tlačítko nahoru: `position: fixed; right: 1rem; bottom: 1rem; z-index: 5; .`
3. Karta v galerii: `position: relative; ; štítek position: absolute; top: 0; left: 0; .`
4. Modal: `position: fixed; inset: 0; background: rgba(0,0,0,.6); display: flex; justify-content: center; align-items: center; z-index: 1000; .`
5. JS: kliknutí na obrázek zavolá `modal.classList.add('open')` , kliknutí na pozadí/X ho zavře.
6. Test překryvů — vyšší `z-index` patří modalu.

KLÍČOVÉ ÚRYVKY KÓDU

 `style.css` — sticky, fixed, modal

```

.header {
  position: sticky;
  top: 0;
  z-index: 10;
  background: #fff;
  box-shadow: 0 2px 6px rgba(0,0,0,.08);
}

.btn-top {
  position: fixed;
  right: 1rem;
  bottom: 1rem;
  z-index: 5;
}

.card {
  position: relative;
}

.card .badge {
  position: absolute;
  top: .5rem;
  left: .5rem;
  background: #d97706;
  color: #fff;
  padding: 2px 8px;
  border-radius: 999px;
}

.modal {
  position: fixed;
  inset: 0;
  background: rgba(0,0,0,.65);
  display: none;
  justify-content: center;
  align-items: center;
  z-index: 1000;
}

.modal.open { display: flex; }
.modal__close {
  position: absolute;
  top: 1rem; right: 1rem;
  background: transparent;
  color: #fff; font-size: 2rem; border: 0;
}

```

```
const modal = document.querySelector('.modal');
const modalImg = modal.querySelector('img');

document.querySelectorAll('.gallery img').forEach(img => {
  img.addEventListener('click', () => {
    modalImg.src = img.src;
    modal.classList.add('open');
  });
});

modal.addEventListener('click', e => {
  if (e.target === modal || e.target.classList.contains('modal__close')) {
    modal.classList.remove('open');
  }
});
```

6 CSS animace a transformace

💡 Elevator pitch

CSS umí prvky plynule přesouvat, otáčet, zvětšovat a měnit barvy bez JavaScriptu. Hlavní nástroje jsou `transform`, `transition` a `@keyframes` s `animation`.

📺 TEORETICKÝ ZÁKLAD

- **Transform funkce:**

- `translate(x, y)` — posun (nezatěžuje layout, GPU akcelerované).
- `rotate(45deg)` — otočení.
- `scale(1.2)` — zvětšení.
- `skew(10deg)` — zkosení.
- Lze řetězit: `transform: translate(10px) rotate(5deg) scale(1.1);`
- `transform-origin` — bod, kolem kterého se transformace děje.

- **Transition** — plynulý přechod mezi dvěma stavy: `transition: all .3s ease;` nebo cíleně `transition: transform .25s ease-out, background .2s;`. Timing functions: `linear`, `ease`, `ease-in`, `ease-out`, `ease-in-out`, `cubic-bezier(...)`.

- **Keyframes — vlastní animace:**

```
@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.1); }
  100% { transform: scale(1); }
}
.btn { animation: pulse 1.5s infinite; }
```

- **Vlastnost animation:** `name duration timing-function delay iteration-count direction fill-mode play-state;`
 - `iteration-count: infinite | n.`
 - `direction: normal | reverse | alternate.`
 - `fill-mode: forwards` — po skončení zachová poslední snímek.
- **Přístupnost:** `@media (prefers-reduced-motion: reduce)` — uživatel nepřeje animace, vypneme.
- **Výkon:** animovat lze cokoliv, ale levné jsou `transform` a `opacity`; ostatní (`width`, `top`, `left`) způsobují relayout.
- **3D:** `perspective`, `rotateY`, `backface-visibility: hidden` pro card flip.

📺 UKÁZKOVÉ ZADÁNÍ

Klíčová slova: transform, transition, @keyframes, animation, hover efekty, prefers-reduced-motion.

Zadání praktického úkolu

Vytvořte stránku „Naše služby“ s animacemi v souborech `index.html` a `style.css` :

1. Tři **karty služeb** v řadě. Při najetí myši se karta plynule zvedne (`translateY -8px`) a vrhne větší stín. Přejechod 0.25 s.
2. **CTA tlačítko** „Objednat“ pomalu pulsuje (`scale 1 → 1.06 → 1`) v nekonečné smyčce 1,5 s.
3. **Card flip**: jedna karta se po najetí překlápí o 180° kolem osy Y a ukáže detail (3D efekt).
4. Při načtení stránky vjede header z horní strany dolů (animace 0.6 s).
5. Respektujte `prefers-reduced-motion: reduce` — animace vypnete.

? Otázky k obhajobě

- Rozdíl mezi `transition` a `animation` ?
- Proč preferovat animaci `transform` před `top/left` ?
- K čemu slouží `fill-mode: forwards` ?

KUCHARKA — POSTUP

1. Karta: `transition: transform .25s ease, box-shadow .25s; + :hover { transform: translateY(-8px); box-shadow: ... } .`
2. CTA: definuji `@keyframes pulse` a aplikuji `animation: pulse 1.5s infinite .`
3. Card flip: rodič `perspective:1000px` , vnitřek `transform-style:preserve-3d;` `transition: transform .6s` ; přední/zadní strana absolutně + `backface-visibility:hidden` ; hover otočí o 180°.
4. Header: `@keyframes slideDown` z `translateY(-100%)` do `translateY(0)` .
5. Reduced motion media query — zruším animace.

KLÍČOVÉ ÚRYVKY KÓDU

 `style.css` — hover lift + pulse + flip

```

/* Lift on hover */
.card {
  transition: transform .25s ease, box-shadow .25s ease;
  box-shadow: 0 2px 6px rgba(0,0,0,.08);
}
.card:hover {
  transform: translateY(-8px);
  box-shadow: 0 12px 24px rgba(0,0,0,.15);
}

/* Pulse CTA */
@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50%      { transform: scale(1.06); }
}
.btn-cta { animation: pulse 1.5s ease-in-out infinite; }

/* Card flip */
.flip { perspective: 1000px; }
.flip__inner {
  position: relative;
  transition: transform .6s;
  transform-style: preserve-3d;
}
.flip:hover .flip__inner { transform: rotateY(180deg); }
.flip__front, .flip__back {
  position: absolute; inset: 0;
  backface-visibility: hidden;
}
.flip__back { transform: rotateY(180deg); }

/* Header slide-in */
@keyframes slideDown {
  from { transform: translateY(-100%); }
  to   { transform: translateY(0); }
}
.header { animation: slideDown .6s ease-out both; }

/* A11y */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation: none !important;
    transition: none !important;
  }
}

```


7 Tabulky v HTML a jejich stylování

Elevator pitch

HTML tabulky slouží k zobrazení strukturovaných dat (řádek × sloupec). Mají vlastní sémantické tagy pro hlavičku, tělo i patičku, podporují slučování buněk a přístupnost pro čtečky.

TEORETICKÝ ZÁKLAD

- **Struktura tabulky:**
 - `<table>` — kontejner.
 - `<caption>` — popisek (zobrazí se nahoře, čtečka přečte).
 - `<thead>` / `<tbody>` / `<tfoot>` — sémantické sekce.
 - `<tr>` — řádek; `<th>` — záhlaví; `<td>` — buňka dat.
 - `<colgroup>` + `<col>` — definice vlastností sloupců (např. šířka, třída).
- **Slučování buněk:**
 - `colspan="2"` — buňka přes 2 sloupce.
 - `rowspan="3"` — buňka přes 3 řádky.
- **Přístupnost:** `<th scope="col">` nebo `scope="row"` ; v komplexních tabulkách `headers="..."` ; `<caption>` je povinné minimum pro screen reader.
- **Stylování CSS:**
 - `border-collapse: collapse;` — sjednocené orámování.
 - `border-spacing` , `caption-side: top|bottom` .
 - Zebra: `tbody tr:nth-child(even) { background: #f6f7fb; }` .
 - Hover na řádek: `tbody tr:hover` .
 - Sticky hlavička: `thead th { position: sticky; top: 0; }` .
- **Responzivita:** obalit do `div` s `overflow-x: auto;` ; alternativně `display:block` + relabelovat sloupce (data-attributes).
- **Tabulky NEPOUŽÍVAT na layouty** — jen na data.

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: table, thead/tbody/tfoot, rowspan, colspan, caption, scope, stylování, responzivita.

Zadání praktického úkolu

Vytvořte přehlednou tabulku „Rozvrh hodin 4. ročníku“ v souborech `index.html` a `style.css`.

1. Tabulka má `<caption>` „Rozvrh — 4.A“.
2. V `<thead>` řádek se dny (Po–Pá), v levém sloupci hodiny (1.–7.).
3. Předmět *Tělesná výchova* ve čtvrtek pokrývá 2 hodiny po sobě → `rowspan="2"`.
4. Předmět *Praxe (DAT)* v pátek pokrývá 3 sloupce, protože je celodenní blok → simulujte sloučením přes `colspan` na samostatném řádku.
5. Stylování: zebra řádky, sticky hlavička při scrollu, hover zvýrazní řádek, na mobilu (max 700 px) tabulku obalte do scrollovatelného containeru.
6. Volné hodiny vyznačte třídou `.empty` a barevně odlište.

? Otázky k obhajobě

- Rozdíl mezi `thead`, `tbody`, `tfoot` — proč je rozlišovat?
- Co dělá atribut `scope` a komu pomáhá?
- Jak se chová `border-collapse`?

KUCHARKA — POSTUP

1. Sestavím sémantickou kostru `<table><caption>...<thead>...<tbody>...</table>`.
2. V `<th>` doplním `scope="col"` resp. `scope="row"`.
3. Pro slučování buněk si na papíře nakreslím mřížku — kde je `rowspan`, vynechám další `<td>` v dalších řádcích.
4. CSS: `border-collapse:collapse`, padding, zebra, hover.
5. Sticky hlavička: `thead th { position:sticky; top:0; background:#fff; }`.
6. Responzivita: wrapper `.table-wrap { overflow-x:auto; }`.

KLÍČOVÉ ÚRYVKY KÓDU

 `index.html` — tabulka se slučováním

```

<div class="table-wrap">
  <table class="schedule">
    <caption>Rozvrh — 4.A</caption>
    <thead>
      <tr>
        <th scope="col">Hod.</th>
        <th scope="col">Po</th>
        <th scope="col">Út</th>
        <th scope="col">St</th>
        <th scope="col">Čt</th>
        <th scope="col">Pá</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <th scope="row">1.</th>
        <td>ČJ</td><td>MA</td><td>AJ</td>
        <td rowspan="2">TV</td>
        <td colspan="3">Praxe (DAT)</td>
      </tr>
      <tr>
        <th scope="row">2.</th>
        <td>ČJ</td><td>FY</td><td>DBS</td>
        <!-- TV pokračuje rowspan, žádné <td> -->
      </tr>
      <tr>
        <th scope="row">3.</th>
        <td>PRG</td><td>PRG</td><td>WEB</td>
        <td class="empty">—</td><td class="empty">—</td><td class="empty">—</td>
      </tr>
    </tbody>
  </table>
</div>

```



style.css — zebra, sticky, hover

```
.table-wrap { overflow-x: auto; }

.schedule {
  width: 100%;
  border-collapse: collapse;
  font-size: 14px;
}
.schedule caption {
  caption-side: top;
  font-weight: 700;
  padding: 8px;
}
.schedule th, .schedule td {
  border: 1px solid #d1d5db;
  padding: 8px 12px;
  text-align: center;
}
.schedule thead th {
  background: #0f766e;
  color: #fff;
  position: sticky;
  top: 0;
}
.schedule tbody tr:nth-child(even) td { background: #f0fdfa; }
.schedule tbody tr:hover td { background: #d6f3ee; }
.schedule .empty { color: #9ca3af; background: #f9fafb; }
```

8 Datové typy a pole

Elevator pitch

Datový typ určuje, jaké hodnoty může proměnná držet a jaké operace nad ní lze provádět. Pole je strukturovaný typ — pojmenovaná posloupnost prvků stejného typu s přístupem podle indexu.

TEORETICKÝ ZÁKLAD

- **Hodnotové typy** (uloženy přímo na zásobníku, kopírují se hodnotou): `int` , `long` , `double` , `decimal` , `bool` , `char` , `struct` , `enum` .
- **Referenční typy** (na haldě, proměnná drží odkaz): `string` , `class` , `array` , `delegate` .
- **Strukturované typy**: `struct` (lehká hodnotová sada polí) vs. `class` (referenční s identitou). Záznam `record` v C# 9+ — neměnný typ s automatickou rovností.
- **Výčtový typ (enum)**:

```
enum Den { Po=1, Ut, St, Ct, Pa, So, Ne }  
Den dnes = Den.St;
```

Interně `int`; lze i `[Flags]` pro bitová pole.

- **Pole (array)** — fixní velikost, indexace od 0:
 - Deklarace: `int[] arr = new int[5];` nebo `int[] arr = {1,2,3};` .
 - Délka: `arr.Length` .
 - Iterace: `for` , `foreach` .
- **Vícerozměrná pole** (rectangular): `int[,] m = new int[3,4];` — matice 3×4. Přístup `m[1,2]` .
- **Jagged (zubatá) pole** — pole polí, řádky mohou mít různou délku: `int[][] j = new int[3][];`
`j[0] = new int[2]; j[1] = new int[5];` .
- **Imutabilita stringu**: `string` v C# je neměnný — operace `s.ToUpper()` vrací nový řetězec.
- **Rozdíl pole vs. List**: pole má fixní velikost a vyšší výkon; `List<T>` dynamicky roste.

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: hodnotové vs. referenční typy, enum, vícerozměrná pole, jagged pole, iterace, vlastní struct.

Zadání praktického úkolu

V konzolové aplikaci v C# (.NET 8) vytvořte program „Klasifikace třídy“ v souboru `Program.cs` :

1. Definujte **enum** `Predmet` s hodnotami {ČJ, MA, AJ, IT}.
2. Vytvořte **struct** `Student` s poli `Jmeno` (string) a `Znamky` — zubaté pole `int[]` (každý student může mít jiný počet známek).
3. V `Main` připravte pole 4 studentů a každému přiřad'te 3–5 známek.
4. Vytvořte **2D pole** `int[,] hodnoceni` rozměru [počet studentů, počet předmětů] a vyplňte ho průměry.
5. Spočítejte a vypište:
 - průměr každého studenta (řádek matice),
 - průměr každého předmětu (sloupec matice),
 - nejlepšího a nejhoršího studenta (podle průměru).

? Otázky k obhajobě

- Rozdíl mezi 2D a jagged polem? Kdy zvolit které?
- Proč jste použili `struct` a ne `class` pro studenta?
- Co je hodnotový a co referenční typ?

KUCHAŘKA — POSTUP

1. Založím konzolovku: `dotnet new console -n Klasifikace` .
2. Definuji `enum Predmet` a `struct Student` .
3. V `Main` vytvořím pole `Student[] studenti` a inicializuji.
4. Naplním 2D pole `int[,] hodnoceni` průměry pro `Predmet` .
5. Smyčky: vnější `for` přes řádky, vnitřní přes sloupce.
6. Pomocné metody `RowAvg(matrix, row)` a `ColAvg(matrix, col)` .
7. Výstup formátuji `string.Format` nebo interpolací `$"..."` .

KLÍČOVÉ ÚRYVKY KÓDU

 `Program.cs` — typy a iterace polí

```

enum Predmet { CJ, MA, AJ, IT }

struct Student
{
    public string Jmeno;
    public int[] Znamky;    // jagged - každý student může mít jiný počet
}

class Program
{
    static void Main()
    {
        var studenti = new Student[]
        {
            new() { Jmeno = "Anna",    Znamky = new[] { 1, 2, 1 } },
            new() { Jmeno = "Petr",    Znamky = new[] { 3, 2, 4, 3 } },
            new() { Jmeno = "Eva",     Znamky = new[] { 1, 1, 2, 1, 1 } },
            new() { Jmeno = "Tomáš",   Znamky = new[] { 4, 5, 3 } },
        };

        // 2D pole: studenti × předměty
        int predmetuCount = Enum.GetValues<Predmet>().Length;
        int[,] hodnoceni = new int[studenti.Length, predmetuCount];

        var rng = new Random(42);
        for (int i = 0; i < studenti.Length; i++)
            for (int j = 0; j < predmetuCount; j++)
                hodnoceni[i, j] = rng.Next(1, 6);

        // Průměry studentů
        for (int i = 0; i < studenti.Length; i++)
        {
            double avg = RowAvg(hodnoceni, i);
            Console.WriteLine($"{studenti[i].Jmeno,-8}: průměr {avg:F2}");
        }

        // Průměry předmětů
        foreach (Predmet p in Enum.GetValues<Predmet>())
            Console.WriteLine($"{p}: {ColAvg(hodnoceni, (int)p):F2}");
    }

    static double RowAvg(int[,] m, int row)
    {
        double sum = 0;
        int cols = m.GetLength(1);
        for (int j = 0; j < cols; j++) sum += m[row, j];
        return sum / cols;
    }
}

```

```
}
```

```
static double ColAvg(int[,] m, int col)
```

```
{
```

```
    double sum = 0;
```

```
    int rows = m.GetLength(0);
```

```
    for (int i = 0; i < rows; i++) sum += m[i, col];
```

```
    return sum / rows;
```

```
}
```

```
}
```


9 Spojové struktury a stromy

Elevator pitch

Spojový seznam je dynamická datová struktura, ve které prvky drží odkaz na sousedy. Strom je hierarchická struktura s jedním kořenem; binární vyhledávací strom umožňuje rychlé vyhledávání v $O(\log n)$.

TEORETICKÝ ZÁKLAD

- **Spojový (lineární) seznam:**
 - **Jednosměrný:** uzel obsahuje data + odkaz `next` na další.
 - **Obousměrný:** uzel má i `prev` — lze procházet oběma směry (.NET `LinkedList<T>`).
 - **Cyklický:** poslední ukazuje na první.
- **Operace a složitost spojáku:**
 - Vložení/odstranění na známém místě: **$O(1)$** .
 - Vyhledání: **$O(n)$** (sekvenční průchod).
 - Pole vs. spoják: pole rychlejší přístup po indexu ($O(1)$), spoják levnější vkládání doprostřed.
- **Strom — pojmy:**
 - *Kořen* (root), *uzel* (node), *list* (leaf), *rodič*, *potomek*, *sourozenec*.
 - *Hloubka* = počet hran od kořene; *výška* = vzdálenost k nejhlubšímu listu.
- **Binární strom** — každý uzel má max. 2 potomky (levý/pravý).
- **BST (binární vyhledávací strom):** v levém podstromu jsou menší, v pravém větší. Hledání/vkládání: $O(\log n)$ průměrně, $O(n)$ ve výškové degeneraci.
- **Vyvážené stromy** — AVL, Red-Black; .NET `SortedDictionary` používá RB strom.
- **Průchody stromem:**
 - **In-order** (L-R-vrchol není před) — typicky tisk BST seřazeně.
 - **Pre-order** (vrchol-L-R) — kopie stromu.
 - **Post-order** (L-R-vrchol) — uvolnění paměti.
 - **Level-order (BFS)** — po vrstvách (fronta).
- **Použití stromů:** indexy v DB (B-tree), souborový systém, AST kompilátoru, DOM, herní AI.

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: spojový seznam, ukazatele, binární vyhledávací strom, vkládání, vyhledávání, in-order průchod, rekurze.

Zadání praktického úkolu

V konzolové aplikaci v C# implementujte vlastní **jednosměrný spojový seznam** a **binární vyhledávací strom** bez použití hotových kolekcí (žádný `List`, `LinkedList`, `SortedSet`):

1. Třída `LinkedList` s metodami `Add(int)`, `Remove(int)`, `Contains(int)`, `Print()`.
2. Třída `BinarySearchTree` s metodami `Insert(int)`, `Contains(int)` a `PrintInOrder()`.
3. V `Main` načtěte 10 čísel od uživatele (nebo z pole) a vložte je do obou struktur.
4. Vypište spoják v pořadí vložení a BST v pořadí *in-order* (čísla budou seřazená vzestupně).
5. Zeptejte se uživatele na číslo a sdělte, zda je v BST přítomné (`Contains`).

? Otázky k obhajobě

- Jaká je složitost vyhledávání v lineárním seznamu vs. v BST?
- Proč in-order průchod BST vrátí seřazená čísla?
- Co se stane, když do BST vložíte řadu 1,2,3,4,5? (degenerace na lineární seznam.)

KUCHARKA — POSTUP

1. Definuji vnitřní třídu `Node` s polem `Value` a `Next` (resp. `Left` / `Right` u stromu).
2. Spoják: udržuji `Head`; `Add` projde na konec, vytvoří nový uzel a nasadí; `Remove` přesune ukazatel `prev.Next = curr.Next`.
3. BST: rekurzivní `Insert(node, value)` — pokud null, vytvoří uzel; pokud hodnota menší → doleva, jinak doprava.
4. In-order: `Print(node.Left); Console.Write(node.Value); Print(node.Right);`.
5. `Contains` v BST: rekurzivně podle porovnání.
6. Test různými vstupy včetně duplikátů a chybějících hodnot.

KLÍČOVÉ ÚRYVKY KÓDU

 `LinkedList.cs` — vlastní jednosměrný seznam

```
public class LinkedList
{
    private class Node
    {
        public int Value;
        public Node? Next;
        public Node(int v) { Value = v; }
    }

    private Node? head;

    public void Add(int value)
    {
        var node = new Node(value);
        if (head == null) { head = node; return; }
        var curr = head;
        while (curr.Next != null) curr = curr.Next;
        curr.Next = node;
    }

    public bool Remove(int value)
    {
        if (head == null) return false;
        if (head.Value == value) { head = head.Next; return true; }
        var curr = head;
        while (curr.Next != null && curr.Next.Value != value) curr = curr.Next;
        if (curr.Next == null) return false;
        curr.Next = curr.Next.Next;
        return true;
    }

    public bool Contains(int value)
    {
        for (var c = head; c != null; c = c.Next)
            if (c.Value == value) return true;
        return false;
    }

    public void Print()
    {
        for (var c = head; c != null; c = c.Next)
            Console.Write(c.Value + " → ");
        Console.WriteLine("null");
    }
}
```



```

public class BinarySearchTree
{
    private class Node
    {
        public int Value;
        public Node? Left, Right;
        public Node(int v) { Value = v; }
    }

    private Node? root;

    public void Insert(int value) ⇒ root = Insert(root, value);

    private Node Insert(Node? node, int value)
    {
        if (node == null) return new Node(value);
        if (value < node.Value) node.Left = Insert(node.Left, value);
        else if (value > node.Value) node.Right = Insert(node.Right, value);
        return node; // duplikát ignoruje
    }

    public bool Contains(int value) ⇒ Contains(root, value);

    private bool Contains(Node? node, int value)
    {
        if (node == null) return false;
        if (value == node.Value) return true;
        return value < node.Value
            ? Contains(node.Left, value)
            : Contains(node.Right, value);
    }

    public void PrintInOrder()
    {
        InOrder(root);
        Console.WriteLine();
    }

    private void InOrder(Node? n)
    {
        if (n == null) return;
        InOrder(n.Left);
        Console.Write(n.Value + " ");
        InOrder(n.Right);
    }
}

```

10 Podprogramy a lambda funkce

Elevator pitch

Podprogram (funkce, procedura, metoda) je pojmenovaný blok kódu se vstupy a výstupem — umožňuje znovupoužití, čitelnost a testování. Lambda je krátký anonymní podprogram, ideální pro předávání chování jako parametru.

TEORETICKÝ ZÁKLAD

- **Podprogram** v C#:
 - Metoda s návratem: `int Plus(int a, int b) { return a+b; }` .
 - Procedura (nic nevrací): `void Vypis(string s) { ... }` .
 - Výrazové tělo: `int Plus(int a, int b) => a+b;` .
- **Předávání parametrů:**
 - **Hodnotou** (default) — kopie.
 - **ref** — odkaz, vyžaduje inicializovanou proměnnou; vstup i výstup.
 - **out** — odkaz, metoda *musí* přiřadit; jen výstup.
 - **in** — read-only odkaz (výkon u velkých struktur).
 - **params** — proměnný počet argumentů.
 - Implicitní hodnoty: `void F(int x = 0)` .
 - Pojmenované argumenty: `F(x: 5)` .
- **Rekurze** — funkce volá sama sebe; potřebuje *základní* a *rekurzivní* krok. Pozor na přetečení zásobníku.
- **Obor platnosti (scope):** proměnné jsou viditelné v bloku `{ }` , ve kterém vznikly.
- **Přetížení (overloading)** — víc metod stejného jména, různé signatury.
- **Lambda výraz** — `(parametry) => výraz` nebo `(parametry) => { příkazy; return v; }` . Anonymní funkce, často přiřazené do `Func<...>` / `Action<...>` .
- **Func vs. Action vs. Predicate:**
 - `Func<T1, ..., TResult>` — funkce vracející hodnotu.
 - `Action<T1, ...>` — procedura (void).
 - `Predicate<T>` — funkce vracející `bool` .
- **Closure** — lambda zachycuje proměnné z okolního scope.
- **LINQ** intenzivně využívá lambdy: `.Where(x => x > 0).Select(x => x*x)` .

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: metoda, parametry (ref/out), přetížení, rekurze, lambda, Func, Predicate, LINQ.

Zadání praktického úkolu

V konzolové aplikaci v C# vytvořte program „Statistika pole“:

1. Metoda `int[] NactiPole(int pocet)` — načte z konzole zadaný počet celých čísel.
2. Metoda `void Statistika(int[] pole, out int min, out int max, out double prumer)` — vrátí 3 hodnoty pomocí `out`.
3. **Rekurzivní** metoda `long Faktorial(int n)` a iterativní `long FaktorialIter(int n)` (přetížení názvy odlišíme).
4. Generická metoda `T[] Filtruj<T>(T[] pole, Predicate<T> podminka)`, kterou zavoláte pro:
 - sudá čísla — lambda `x ⇒ x % 2 == 0`,
 - čísla větší než průměr — lambda zachycující průměr (closure).
5. Vypište výsledky pomocí `Action<int[]>` uloženého v proměnné `tisk`.

? Otázky k obhajobě

- Rozdíl mezi `ref` a `out` parametrem?
- Co je closure a kdy nastává?
- Proč lambda `x ⇒ x*x` nemá deklarovaný typ?

KUCHAŘKA — POSTUP

1. Načtu vstup metodou s `for` a `int.Parse(Console.ReadLine())`.
2. Implementuji statistiku přes `out` parametry — každému musím v metodě přiřadit hodnotu.
3. Rekurse: `n ≤ 1 ? 1 : n * Faktorial(n-1);`.
4. Generická filtrace: vytvořím `List<T>`, projdu vstupní pole a přidám prvek, pokud lambda vrátí true; pak `.ToArray()`.
5. Average spočítám zvlášť, předám do filtru jako closure.
6. Tisk: `Action<int[]> tisk = a ⇒ Console.WriteLine(string.Join(", ", a));`.

KLÍČOVÉ ÚRYVKY KÓDU

 Program.cs — out parametry, rekurze, generická lambda

```

class Program
{
    static int[] NactiPole(int pocet)
    {
        var pole = new int[pocet];
        for (int i = 0; i < pocet; i++)
        {
            Console.Write($"{i+1}: ");
            pole[i] = int.Parse(Console.ReadLine());
        }
        return pole;
    }

    static void Statistika(int[] p, out int min, out int max, out double avg)
    {
        min = p[0]; max = p[0]; long sum = 0;
        foreach (var x in p)
        {
            if (x < min) min = x;
            if (x > max) max = x;
            sum += x;
        }
        avg = (double)sum / p.Length;
    }

    static long Faktorial(int n) => n <= 1 ? 1 : n * Faktorial(n - 1);

    static T[] Filtruj<T>(T[] pole, Predicate<T> podm)
    {
        var list = new List<T>();
        foreach (var x in pole) if (podm(x)) list.Add(x);
        return list.ToArray();
    }

    static void Main()
    {
        int[] data = NactiPole(5);
        Statistika(data, out int min, out int max, out double avg);
        Console.WriteLine($"Min={min}, Max={max}, Průměr={avg:F2}");

        Console.WriteLine("5! = " + Faktorial(5));

        // Sudá čísla
        var suda = Filtruj(data, x => x % 2 == 0);
        // Closure – lambda zachytí avg
        var nadprumer = Filtruj(data, x => x > avg);
    }
}

```



```
    Action<int[]> task = a => Console.WriteLine(string.Join(", ", a));  
    task(suda);  
    task(nadprumer);  
}  
}
```

11 Kolekce: pole, zásobník, fronta, slovník

Elevator pitch

Kolekce jsou hotové datové struktury v `System.Collections.Generic`. Volba správné kolekce zásadně ovlivňuje výkon: pole pro pevnou velikost, Stack pro LIFO, Queue pro FIFO, Dictionary pro rychlé hledání podle klíče.

TEORETICKÝ ZÁKLAD

- **Pole (Array)** — fixní velikost, $O(1)$ přístup po indexu.
- **List<T>** — dynamicky se zvětšuje, vnitřně pole. `Add` : amortizovaně $O(1)$; `Insert(0, ...)` : $O(n)$.
- **Stack<T> (zásobník, LIFO)** — Last In First Out. Operace `Push`, `Pop`, `Peek`. Použití: undo/redo, parser závorek, volání metod.
- **Queue<T> (fronta, FIFO)** — First In First Out. Operace `Enqueue`, `Dequeue`, `Peek`. Použití: tiskové úlohy, BFS, zprávy mezi vlákny.
- **Dictionary<TKey, TValue> (slovník, hash-mapa)** — klíč → hodnota, průměrně $O(1)$ lookup. Klíč musí mít kvalitní `GetHashCode`.
- **HashSet<T>** — množina unikátních prvků, rychlé `Contains`.
- **SortedDictionary / SortedSet** — vždy seřazené (RB strom), $O(\log n)$.
- **LinkedList<T>** — obousměrně spojený seznam.
- **Iterace:** všechny implementují `IEnumerable<T>` → `foreach`.
- **Composite operace** přes LINQ: `.Where`, `.Select`, `.GroupBy`, `.OrderBy`, `.ToDictionary`, ...

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: Stack (LIFO), Queue (FIFO), Dictionary, HashSet, volba správné struktury, výkon.

Zadání praktického úkolu

V konzolové aplikaci v C# vytvořte program „Helpdesk simulátor“, který demonstruje všechny čtyři kolekce:

1. **Stack:** Implementujte funkci `JsouZavorkySpravne(string vyraz)`, která ověří, zda jsou závorky `()`, `[]`, `{}` ve výrazu správně spárované.
2. **Queue:** Vytvořte frontu úkolů. Uživatel může `add Petr`, `next` (vytáhne první v pořadí) nebo `list`.
3. **Dictionary:** Ved'te počty tiketů na uživatele — klíč je jméno, hodnota počet (`+= 1` při každém `add`).
4. **HashSet:** Při pokusu vložit již existujícího uživatele do seznamu VIP klientů ho neduplikujte; vrátí `false` a vypíše varování.
5. Hlavní program běží v cyklu, dokud uživatel nezadá `quit`.

? Otázky k obhajobě

- Co je LIFO a FIFO — uveďte reálný příklad.
- Proč je hledání v Dictionary v průměru $O(1)$?
- Kdy zvolit HashSet místo List?

KUCHARKA — POSTUP

1. Stack: pro každý znak `(/[/{ push; pro )/]/}` kontrola párování s `Pop`; nakonec stack musí být prázdný.
2. Queue: vstup parsuju `switch` em na příkazy.
3. Dictionary: `if (counts.TryGetValue(name, out var c)) counts[name] = c+1; else counts[name] = 1;` nebo přes `CollectionsMarshal.GetValueRefOrAddDefault`.
4. HashSet: `vip.Add(name)` vrátí `false`, když už existuje.
5. Hlavní smyčku ukončím `break` na `quit`.

KLÍČOVÉ ÚRYVKY KÓDU

 Závorky přes Stack

```
static bool JsouZavorkySpravne(string s)
{
    var pairs = new Dictionary<char, char> { [')']='(', ['']']='[', ['}']='{' };
    var stack = new Stack<char>();
    foreach (var c in s)
    {
        if (c == '(' || c == '[' || c == '{') stack.Push(c);
        else if (pairs.ContainsKey(c))
        {
            if (stack.Count == 0 || stack.Pop() != pairs[c]) return false;
        }
    }
    return stack.Count == 0;
}
```



 Hlavní smyčka — fronta + slovník + množina

```
var fronta = new Queue<string>();
var pocty = new Dictionary<string, int>();
var vip = new HashSet<string>();

while (true)
{
    Console.Write("> ");
    var line = Console.ReadLine();
    if (line == null || line == "quit") break;
    var parts = line.Split(' ', 2);
    var cmd = parts[0];

    switch (cmd)
    {
        case "add" when parts.Length == 2:
            var name = parts[1];
            fronta.Enqueue(name);
            pocty[name] = pocty.TryGetValue(name, out var c) ? c + 1 : 1;
            Console.WriteLine($"Přidán {name} (tiketů: {pocty[name]})");
            break;
        case "next":
            if (fronta.Count == 0) Console.WriteLine("Fronta prázdná.");
            else Console.WriteLine("Obsluhujeme: " + fronta.Dequeue());
            break;
        case "list":
            Console.WriteLine(string.Join(" → ", fronta));
            break;
        case "vip" when parts.Length == 2:
            if (!vip.Add(parts[1])) Console.WriteLine("Už je VIP.");
            else Console.WriteLine("VIP přidán.");
            break;
    }
}
```

12 Souborový systém a streamy

Elevator pitch

Souborový systém je hierarchická struktura adresářů a souborů. Stream je abstrakce posloupnosti bytů — používáme ho ke čtení i zápisu souborů, síti i paměti. .NET nabízí třídy `File`, `FileStream`, `StreamReader/Writer`.

TEORETICKÝ ZÁKLAD

- **Souborový systém:** kořen (Windows: `C:\`; Linux: `/`), adresáře, soubory; absolutní vs. relativní cesta.
- **Cesty v .NET:** `Path.Combine`, `Path.GetFileName`, `Path.GetExtension`, `Path.GetFullPath`; oddělovač `Path.DirectorySeparatorChar`.
- **Třídy `File` a `Directory`** (statické):
 - `File.Exists`, `Delete`, `Move`, `Copy`, `ReadAllText`, `WriteAllText`, `ReadAllLines`, `AppendAllText`.
 - `Directory.CreateDirectory`, `GetFiles`, `GetDirectories`.
- **Streamy** — `Stream` abstraktní třída, dědí `FileStream`, `MemoryStream`, `NetworkStream`.
- **Textové vs. binární:**
 - Text: `StreamReader` / `StreamWriter` obtéká stream a pracuje s kódováním (UTF-8 default).
 - Binární: `BinaryReader` / `BinaryWriter` — čte/píše typy (`int`, `double`) v binárním formátu.
- **Režimy otevření:** `FileMode.Open`, `Create`, `Append`; `FileAccess.Read/Write`; `FileShare`.
- **using / IDisposable** — zaručí uvolnění zdrojů (uzavření streamu) i při výjimce. Doporučeno místo manuálního `Close`.
- **Asynchronní práce:** `ReadAsync`, `WriteAsync`, `File.ReadAllTextAsync` — pro velké soubory a I/O.
- **CSV/JSON/XML:**
 - CSV ručně přes `Split` nebo knihovnu `CsvHelper`.
 - JSON: `System.Text.Json` (`JsonSerializer.Serialize/Deserialize`).
 - XML: `System.Xml`, LINQ to XML.

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: File, StreamReader/Writer, using, FileMode, JSON serializace, výjimky při I/O.

Zadání praktického úkolu

V konzolové aplikaci v C# vytvořte program, který zpracuje seznam zákazníků v souboru `customers.csv` a vyrobí JSON souhrn:

1. Načtěte vstupní soubor řádek po řádku přes `StreamReader` (každý řádek: `jmeno;email;vek` ; první řádek je hlavička, přeskočte ji).
2. Pro každý řádek vytvořte instanci `record Customer(string Jmeno, string Email, int Vek)` a přidejte ji do `List` .
3. Spočítejte průměrný věk a počet zákazníků starších 18 let.
4. Souhrn (`{"pocet":N, "prumer":X, "dospelych":Y, "zakaznici":[ ... ]}`) zapište do `summary.json` pomocí `JsonSerializer` .
5. Ošetřete výjimky: chybějící soubor (`FileNotFoundException`), špatný formát řádku — vypište přátelskou hlášku.
6. Použijte `using` bloky pro všechny streamy.

? Otázky k obhajobě

- K čemu slouží `using` a co se stane bez něj?
- Rozdíl mezi `StreamReader` a `BinaryReader` ?
- Proč preferovat `File.ReadAllTextAsync` u velkých souborů?

KUCHARKA — POSTUP

1. Definuji `record Customer(string Jmeno, string Email, int Vek);` .
2. `using var sr = new StreamReader("customers.csv");` — čtu `sr.ReadLine()` ve smyčce.
3. Hlavičku přeskočím, zbytek `line.Split(';') + int.Parse` pro věk.
4. Statistiku: `list.Average(c => c.Vek)` , `list.Count(c => c.Vek >= 18)` .
5. Serializace: `JsonSerializer.Serialize(obj, new JsonSerializerOptions { WriteIndented = true })` .
6. Zapišu přes `File.WriteAllText("summary.json", json)` .
7. Celé zabalím do `try/catch` na `FileNotFoundException` , `FormatException` .

KLÍČOVÉ ÚRYVKY KÓDU

 Program.cs — CSV → objekt → JSON

```

using System.Text.Json;

record Customer(string Jmeno, string Email, int Vek);

try
{
    var list = new List<Customer>();
    using (var sr = new StreamReader("customers.csv"))
    {
        sr.ReadLine(); // přeskočit hlavičku
        string? line;
        while ((line = sr.ReadLine()) != null)
        {
            var p = line.Split(';');
            if (p.Length != 3) { Console.WriteLine($"Špatný řádek: {line}"); continue; }
            list.Add(new Customer(p[0], p[1], int.Parse(p[2])));
        }
    }

    var souhrn = new
    {
        pocet = list.Count,
        prumer = list.Count == 0 ? 0 : list.Average(c => c.Vek),
        dospelych = list.Count(c => c.Vek >= 18),
        zakaznici = list.Count
    };

    var json = JsonSerializer.Serialize(souhrn,
        new JsonSerializerOptions { WriteIndented = true });
    File.WriteAllText("summary.json", json);

    Console.WriteLine($"Zapsáno {list.Count} zákazníků do summary.json");
}
catch (FileNotFoundException)
{
    Console.WriteLine("Soubor customers.csv nebyl nalezen.");
}
catch (FormatException ex)
{
    Console.WriteLine("Špatný formát čísla: " + ex.Message);
}

```


13 Paralelní a asynchronní programování

Elevator pitch

Paralelní programování spouští více úloh najednou na různých vláknech (CPU). Asynchronní programování naopak nečeká na pomalou operaci (sít, disk) — uvolní vlákno a zpracuje výsledek, až je hotový. V .NET je klíčový vzor `async/await` + `Task`.

TEORETICKÝ ZÁKLAD

- **Vlákno (Thread)** — nejnižší úroveň: `new Thread(...).Start()`. Drahé, manuální správa. Dnes spíš výjimečně.
- **Task** — abstraktnější jednotka práce, běží na thread pool. `Task.Run(() => ...)`.
- **async / await** — syntaxe pro asynchronní kód, který se čte sekvenčně. Metoda označená `async` vrací `Task` nebo `Task<T>`.
- **Rozdíl async vs. paralelní:**
 - *Asynchronní* = nezdržovat vlákno čekáním (I/O bound — síť, DB, soubor).
 - *Paralelní* = více práce naráz (CPU bound — výpočet).
- **Task.WhenAll** — čeká na dokončení všech úloh; **Task.WhenAny** — na první.
- **Parallel.For / Parallel.ForEach** — datová paralelizace; `PLINQ : data.AsParallel().Where(...)`.
- **Synchronizace:** `lock (obj) { ... }`, `Monitor`, `SemaphoreSlim`, `Interlocked.Increment`.
- **Race condition** — nedeterministické chování při souběžném přístupu k sdílenému stavu.
- **Deadlock** — vlákna se vzájemně čekají na zámky.
- **Předávání dat mezi vlákny:** návratová hodnota `Task<T>`, `BlockingCollection`, `Channel<T>` (moderní), `ConcurrentQueue/Dictionary`.
- **CancellationToken** — kooperativní zrušení dlouhotrvající operace.
- **ConfigureAwait(false)** — pro knihovny, aby pokračování neběželo na původním synchronizačním kontextu.

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: `async/await`, `Task`, `Task.WhenAll`, `Parallel.For`, `HttpClient`, `CancellationToken`, `race condition`.

Zadání praktického úkolu

V konzolové aplikaci v C# vytvořte program „Web Pinger“, který paralelně ověří dostupnost zadaných URL:

1. Pole 5 URL (např. `https://example.com` , `https://www.google.com` , `https://invalid.invalid`).
2. Asynchronní metoda `Task<PingResult> PingAsync(string url, CancellationToken ct)` používající `HttpClient.GetAsync` ; měřte dobu (`Stopwatch`) a vraťte status kód, čas a chybu.
3. V `Main` spusťte všech 5 pingů paralelně přes `Task.WhenAll` a vypište souhrn.
4. Implementujte timeout 5 sekund přes `CancellationTokenSource` .
5. Pomocí `Parallel.For` spočítejte součet čtverců prvních 1 000 000 čísel a porovnejte výsledek s sekvenčním řešením (zda je shodný).
6. Použijte `Interlocked.Add` pro thread-safe sčítání nebo `Parallel.For` přetížení s `localInit/localFinally` .

? Otázky k obhajobě

- Co dělá klíčové slovo `await` ?
- Kdy použít paralelní a kdy asynchronní přístup?
- Co je `race condition` a jak jí předejít?

KUCHARKA — POSTUP

1. `HttpClient` jako `static readonly` (jeden pro celou aplikaci).
2. Pro každou URL vytvořím `Task<PingResult>` a uložím do pole; `await Task.WhenAll(tasks);` .
3. Uvnitř `PingAsync` : `Stopwatch.StartNew()` , `await client.GetAsync(url, ct)` , do `try/catch` zachytím `HttpRequestException` i `OperationCanceledException` .
4. Timeout: `using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5));` .
5. Paralelní suma: `Parallel.For(0, n, () => 0L, (i, _, local) => local + i * (long)i, local => Interlocked.Add(ref sum, local));` .
6. Výpis výsledků: tabulka URL → status, čas.

📖 KLÍČOVÉ ÚRYVKY KÓDU

📄 async/await + Task.WhenAll

```
using System.Diagnostics;

record PingResult(string Url, int? Status, long Ms, string? Error);

static readonly HttpClient http = new();

static async Task<PingResult> PingAsync(string url, CancellationToken ct)
{
    var sw = Stopwatch.StartNew();
    try
    {
        var res = await http.GetAsync(url, ct);
        sw.Stop();
        return new PingResult(url, (int)res.StatusCode, sw.ElapsedMilliseconds,
    }
    catch (Exception ex)
    {
        sw.Stop();
        return new PingResult(url, null, sw.ElapsedMilliseconds, ex.Message);
    }
}

static async Task Main()
{
    var urls = new[]
    {
        "https://example.com",
        "https://www.google.com",
        "https://invalid.invalid"
    };

    using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5));
    var tasks = urls.Select(u => PingAsync(u, cts.Token));
    var results = await Task.WhenAll(tasks);

    foreach (var r in results)
        Console.WriteLine($"{r.Url,-30} {r.Status?.ToString() ?? "ERR",4} {r.Ms,
}
```

📄 Parallel.For s thread-safe agregací

```
long sum = 0;
Parallel.For<long>(
    fromInclusive: 0,
    toExclusive: 1_000_000,
    localInit: () => 0L,
    body: (i, state, local) => local + (long)i * i,
    localFinally: local => Interlocked.Add(ref sum, local)
);
Console.WriteLine($"Σ i² = {sum}");
```

14 Verzovací systémy: Git a GitHub

Elevator pitch

Git je distribuovaný verzovací systém — sleduje změny v projektu, umožňuje vracet se v historii, větvit a slučovat práci více lidí. GitHub je nejpopulárnější hosting Git repozitářů s nadstavbou (PR, issues, CI/CD).

TEORETICKÝ ZÁKLAD

- **VCS (Version Control System)** — nástroj pro sledování verzí souborů.
- **Centralizované vs. distribuované:** CVS/SVN (jeden server) × Git (každý klient má kompletní historii).
- **Tři oblasti Gitu:**
 - **Working directory** — pracovní soubory.
 - **Staging (index)** — co půjde do dalšího commitu (`git add`).
 - **Repository (.git)** — historie commitů.
- **Commit** — neměnitelný snímek + autor + zpráva + hash (SHA-1).
- **Branch** — pohyblivý ukazatel na commit; větvení odděluje práci.
- **HEAD** — ukazatel na aktuální commit/větev.
- **Merge vs. rebase:**
 - `merge` — vytvoří merge commit, zachová historii větvení.
 - `rebase` — přepíše commity na nový base, lineární historie.
- **Konflikt** nastane, když se dva commity dotknou stejných řádků; Git označí v souboru a čeká na ruční řešení.
- **Remote** — vzdálený repozitář (`origin`). Operace: `clone` , `fetch` , `pull` (= fetch + merge), `push` .
- **GitHub flow:** 1. branch z main, 2. commits, 3. push, 4. Pull Request, 5. review + CI, 6. merge.
- **.gitignore** — soubory, které Git ignoruje (bin/, node_modules/, .env).
- **Tagy** — pojmenované verze (`v1.0.0`).
- **Reset/Revert/Checkout:**
 - `git revert <sha>` — vytvoří nový commit, který ruší změny (bezpečné).
 - `git reset --soft|--mixed|--hard` — posune ukazatel větve (přepisuje historii).
 - `git checkout <branch>` nebo `git switch` , `git restore` .

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: init, add, commit, branch, merge, push, pull, remote, .gitignore, GitHub PR, konflikt.

Zadání praktického úkolu

Předved'te v terminálu (Git Bash / PowerShell) následující workflow. Po každém kroku vypište `git status` resp. `git log --online --all --graph`, abyste mohli komentovat stav.

1. Inicializujte nový repozitář `todo-app` v lokálním adresáři. Přidejte soubor `.gitignore` ignorující `bin/`, `obj/`, `*.log`.
2. Vytvořte `index.html` a `app.js` s minimálním obsahem. Proved'te první commit s vhodnou zprávou v imperativu.
3. Vytvořte větev `feature/add-form`, přidejte do `index.html` formulář pro nový úkol a v `app.js` obsluhu události. Commitněte na nové větví.
4. Přepněte zpět na `main`, na něm proved'te jiný commit (např. úprava CSS) — záměrně tím vytvoříte rozdělení historie.
5. Sluňte větev `feature/add-form` do `main` přes `git merge`. Pokud nastane konflikt, vyřešte jej a dokončete merge.
6. Vytvořte na GitHubu prázdný repozitář `todo-app`, přidejte ho jako `origin` a pushněte všechny větve i tagy.
7. Označte commit tagem `v0.1.0` a pushněte.

? Otázky k obhajobě

- Rozdíl mezi `git pull` a `git fetch`?
- Co dělá `git rebase` a kdy ho preferovat před `merge`?
- Jak vrátit poslední commit, který je už pushnutý?

KUCHARKA — POSTUP

1. `git init` + `git config user.name/email`.
2. Soubory `.gitignore`, `index.html`, `app.js`; `git add .`; `git commit -m "Initial commit"`.
3. `git switch -c feature/add-form`; úpravy; `commit`.
4. `git switch main`; změna v CSS; `commit`.
5. `git merge feature/add-form` — případný konflikt vyřešit, `git add`, `git commit`.
6. Na GitHubu repo bez README; lokálně `git remote add origin URL`; `git push -u origin main`; `git push --all`; `git push --tags`.
7. `git tag v0.1.0`; `git push origin v0.1.0`.

KLÍČOVÉ ÚRYVKY KÓDU

Typické příkazy

```
# Inicializace a první commit
git init
echo "bin/\`nobj/\`n*.log" > .gitignore
git add .
git commit -m "Initial scaffold"

# Větvení
git switch -c feature/add-form
# úpravy...
git add index.html app.js
git commit -m "Add task form and submit handler"

# Merge zpět do main
git switch main
git merge feature/add-form          # případně vyřešit konflikt

# Spojení s GitHubem
git remote add origin https://github.com/uzivatel/todo-app.git
git push -u origin main
git push --all
git tag -a v0.1.0 -m "First public version"
git push origin v0.1.0
```

Vrácení / oprava

```
# Zruší poslední commit, ponechá změny ve stagingu
git reset --soft HEAD~1

# Vrátí konkrétní commit (přidá nový reverzní commit) – bezpečné u sdílené větve
git revert <sha>

# Nahradí pracovní soubor obsahem z posledního commitu
git restore index.html
```



 Elevator pitch

ER (Entity–Relationship) model je grafický návrh databáze ještě před napsáním SQL. Zachycuje entity (tabulky), jejich atributy a vztahy mezi nimi. Z ER diagramu se odvodí relační schéma.

 TEORETICKÝ ZÁKLAD

- **Entita** — objekt reálného světa, který chceme uchovat (Zákazník, Objednávka). V relačním modelu = tabulka.
- **Atribut** — vlastnost entity (jméno, datum). V tabulce = sloupec. Druhy: jednoduchý, složený (jméno + příjmení), vícehodnotový (telefony — porušuje 1NF), odvozený (věk z data narození).
- **Klíče:**
 - **Kandidátní klíč** — sloupec/skupina, která jednoznačně identifikuje záznam.
 - **Primární klíč (PK)** — vybraný kandidátní klíč; **NOT NULL** a unikátní.
 - **Cizí klíč (FK)** — odkaz na PK jiné tabulky, zajišťuje vazbu.
 - **Surrogate klíč** — uměle vygenerované ID (auto-increment, GUID).
 - **Složený klíč** — PK z více sloupců (typické u vazebních tabulek N:M).
- **Relace (vztah)** mezi entitami; **kardinalita:**
 - **1:1** — Uživatel ↔ Profil (málo časté, často sloučíme do 1 tabulky).
 - **1:N** — Zákazník → Objednávky (FK na straně N).
 - **N:M** — Studenti × Předměty; rozkládá se na *vazební tabulku* se 2 FK.
- **Účast:** povinná (každá objednávka *musí* mít zákazníka) vs. volitelná.
- **ER notace:**
 - Klasická Chen: entita = obdélník, atribut = ovál, vztah = kosočtverec.
 - Crow's foot („vrabčí stopa“) — preferovaná v praxi: čára s „nožkou“ na straně N.
- **Slabá entita** — bez vlastního PK, identifikuje se přes vztah k silné (např. položka objednávky).
- **Integritní omezení:** doménová (datový typ, CHECK), entitní (PK), referenční (FK), uživatelská (business pravidla).
- **Návrh krok za krokem:**
 1. Sběr požadavků.
 2. Identifikace entit a atributů.
 3. Určení vztahů a kardinalit.
 4. Volba PK, doplnění FK.
 5. Normalizace (1NF, 2NF, 3NF).
 6. Převod do **CREATE TABLE**.

TÉMA Č. 15 — ER MODEL A NÁVRH DATABÁZE

Klíčová slova: entita, atribut, kandidátní/primární/cizí klíč, kardinalita 1:1/1:N/N:M, vazební tabulka, integrita.

Zadání praktického úkolu

Navrhněte databázi pro malou knihovnu:

1. Identifikujte entity (minimálně *Kniha*, *Autor*, *Čtenář*, *Výpůjčka*) a u každé uveďte atributy (PK kurzívou).
2. Určete vztahy a jejich kardinalitu:
 - jeden *Čtenář* může mít více *Výpůjček*, jedna *Výpůjčka* patří právě jednomu čtenáři (1:N),
 - jedna *Kniha* může mít více *Autorů* a jeden autor více knih (N:M → vazební tabulka),
 - jedna *Kniha* může být v mnoha výpůjčkách (1:N).
3. Nakreslete ER diagram (Crow's foot).
4. Zapište výsledné schéma jako **CREATE TABLE** v MySQL/PostgreSQL syntaxi včetně PK, FK a vhodných omezení.
5. Napište 3 příklady, jak by se z této databáze získalo: (a) výpůjčky čtenáře „Novák“, (b) seznam knih daného autora, (c) knihy aktuálně nevrácené.

? Otázky k obhajobě

- Proč rozkládáme N:M do vazební tabulky?
- Co je referenční integrita a co dělá **ON DELETE CASCADE** ?
- Rozdíl mezi kandidátním a primárním klíčem?

 KUCHAŘKA — POSTUP

1. Vyjmenuji entity, u každé sepíšu atributy.
2. Označím PK (typicky surrogate **id INT AUTO_INCREMENT**).
3. Pro 1:N přidám FK na straně N.
4. Pro N:M vytvořím vazební tabulku se složeným PK z obou FK.
5. Doplním **NOT NULL** , **UNIQUE** , **CHECK** tam, kde je to logické.
6. Nakreslím Crow's foot diagram.
7. Přepíšu do SQL (**CREATE TABLE** v pořadí: nejdřív tabulky bez FK, pak s FK).

 KLÍČOVÉ ÚRYVKY KÓDU

 **schema.sql** — relační schéma knihovny

-- 1) Tabulky bez závislostí

```
CREATE TABLE Autor (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    jmeno         VARCHAR(50) NOT NULL,  
    prijmeni      VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE Kniha (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    isbn          VARCHAR(17) UNIQUE,  
    nazev         VARCHAR(150) NOT NULL,  
    rok_vydani    YEAR,  
    pocet_kusu    INT NOT NULL DEFAULT 1 CHECK (pocet_kusu ≥ 0)  
);
```

```
CREATE TABLE Ctenar (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    jmeno         VARCHAR(50) NOT NULL,  
    prijmeni      VARCHAR(50) NOT NULL,  
    email         VARCHAR(120) UNIQUE  
);
```

-- 2) Vazební tabulka N:M (Kniha-Autor)

```
CREATE TABLE KnihaAutor (  
    kniha_id     INT NOT NULL,  
    autor_id     INT NOT NULL,  
    PRIMARY KEY (kniha_id, autor_id),  
    FOREIGN KEY (kniha_id) REFERENCES Kniha(id) ON DELETE CASCADE,  
    FOREIGN KEY (autor_id) REFERENCES Autor(id) ON DELETE CASCADE  
);
```

-- 3) 1:N (Ctenar → Vypujcka), 1:N (Kniha → Vypujcka)

```
CREATE TABLE Vypujcka (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    ctenar_id     INT NOT NULL,  
    kniha_id      INT NOT NULL,  
    datum_od      DATE NOT NULL,  
    datum_do      DATE,  
    vraceno       BOOLEAN NOT NULL DEFAULT FALSE,  
    FOREIGN KEY (ctenar_id) REFERENCES Ctenar(id) ON DELETE RESTRICT,  
    FOREIGN KEY (kniha_id) REFERENCES Kniha(id) ON DELETE RESTRICT  
);
```

```
-- (a) Výpůjčky čtenáře Novák
SELECT k.nazev, v.datum_od, v.vraceno
FROM   Vypujcka v
JOIN    Ctenar c   ON c.id = v.ctenar_id
JOIN    Kniha  k    ON k.id = v.kniha_id
WHERE   c.prijmeni = 'Novák';

-- (b) Knihy autora "J. K. Rowling"
SELECT k.nazev
FROM   Kniha k
JOIN    KnihaAutor ka ON ka.kniha_id = k.id
JOIN    Autor a       ON a.id = ka.autor_id
WHERE   a.jmeno = 'J. K.' AND a.prijmeni = 'Rowling';

-- (c) Aktuálně nevrácené knihy
SELECT k.nazev, c.prijmeni, v.datum_od
FROM   Vypujcka v
JOIN    Kniha  k ON k.id = v.kniha_id
JOIN    Ctenar c ON c.id = v.ctenar_id
WHERE   v.vraceno = FALSE;
```

16 SQL — výběr a filtrování dat

Elevator pitch

SQL `SELECT` je hlavní nástroj pro získávání dat z relační databáze. Pomocí `JOIN` , `WHERE` , `GROUP BY` , `HAVING` a `ORDER BY` z dat vyfiltrujeme přesně to, co potřebujeme.

TEORETICKÝ ZÁKLAD

- Šablona:

```
SELECT  sloupce
FROM    tabulka
JOIN    jina ON podminka
WHERE   filtr_radku
GROUP BY sloupec
HAVING  filtr_skupin
ORDER BY sloupec [ASC|DESC]
LIMIT  n OFFSET m;
```

- **Logické pořadí provádění:** FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
- **WHERE** — operátory:
 - `=` , `<>` , `<` , `>` , `≤` , `≥`
 - `BETWEEN a AND b`
 - `IN (1,2,3)`
 - `LIKE '%xyz%'` (`%` jakákoli posloupnost, `_` jeden znak)
 - `IS NULL` / `IS NOT NULL` (NULL nelze porovnávat `=`)
 - `AND` , `OR` , `NOT`
- **JOINy:**
 - `INNER JOIN` — průnik (jen řádky se shodou).
 - `LEFT JOIN` — všechny z levé + shoda z pravé (jinak NULL).
 - `RIGHT JOIN` — naopak.
 - `FULL OUTER JOIN` — všechny z obou.
 - `CROSS JOIN` — kartézský součin.
 - Self join — tabulka se sebou (zaměstnanec a jeho šéf).
- **Agregační funkce:** `COUNT(*)` , `SUM` , `AVG` , `MIN` , `MAX` — kombinují se s `GROUP BY` .
- **HAVING vs. WHERE:** `WHERE` filtruje řádky před agregací, `HAVING` skupiny po ní.
- **Aliases:** `SELECT u.jmeno AS uzivatel , FROM Uzivatel u .`
- **DISTINCT** — eliminuje duplicity.

- **Subquery (poddotaz):** v `WHERE ( WHERE id IN (SELECT ...) ), v FROM (derived table) i v SELECT` .
- **CTE (Common Table Expression):** `WITH t AS (...) SELECT * FROM t` — pojmenované poddotazy, čitelnější.
- **Stránkování:** `LIMIT n OFFSET m` (MySQL/Postgres) nebo `OFFSET FETCH` (MS SQL).
- **Funkce nad sloupci:** `UPPER` , `LOWER` , `LENGTH` , `CONCAT` , `YEAR(date)` , `NOW()` , `COALESCE(a,b)` (první nennul).

UKÁZKOVÉ ZADÁNÍ

TÉMA Č. 16 — SQL VÝBĚR DAT

Klíčová slova: SELECT, JOIN, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT, agregace, poddotazy, aliasy.

Zadání praktického úkolu

Máte k dispozici databázi e-shopu se třemi tabulkami: `Zakaznik(id, jmeno, prijmeni, mesto)` , `Objednavka(id, zakaznik_id, datum, castka)` , `Polozka(objednavka_id, produkt, mnozstvi, cena)` .

Napište SQL dotazy, které:

1. Vrábí všechny zákazníky z města „Praha“ seřazené podle příjmení.
2. Spočítají celkovou tržbu (SUM částek) v roce 2025.
3. Najdou top 5 zákazníků podle obrátu — výstup: jméno, příjmení, suma.
4. Vrábí seznam zákazníků, kteří v roce 2025 *nic neobjednali* (LEFT JOIN + IS NULL).
5. Najdou produkty, které se prodaly více než 100×; výstup: produkt, suma kusů.
6. Pro každé město vypíše průměrnou hodnotu objednávky a počet zákazníků.
7. (Bonus) Pomocí CTE najdou objednávky, jejichž částka je nad průměrem všech objednávek.

? Otázky k obhajobě

- Rozdíl mezi `WHERE` a `HAVING` ?
- Co dělá `LEFT JOIN` oproti `INNER JOIN` ?
- Proč `WHERE jmeno = NULL` nefunguje?

KUCHARKA — POSTUP

1. Vždycky začnu od `FROM` a postupně přidávám JOINy.
2. Pro filtrování řádků `WHERE` , pro agregaci `GROUP BY` , pro filtr po agregaci `HAVING` .
3. U seskupení vybírám jen sloupce, které jsou ve `GROUP BY` nebo agregaci.
4. Top N: `ORDER BY ... DESC LIMIT N` .
5. Negativní seznamy (kdo NEMÁ): `LEFT JOIN + IS NULL` nebo `NOT EXISTS` .
6. Nečitelný dotaz vyřeším CTE/aliasy.

KLÍČOVÉ ÚRYVKY KÓDU

 queries.sql — řešení 1–7

```

-- 1) Pražští zákazníci
SELECT id, jmeno, prijmeni
FROM   Zakaznik
WHERE  mesto = 'Praha'
ORDER  BY prijmeni;

-- 2) Tržba 2025
SELECT SUM(castka) AS trzba_2025
FROM   Objednavka
WHERE  YEAR(datum) = 2025;

-- 3) Top 5 zákazníků podle obratu
SELECT z.jmeno, z.prijmeni, SUM(o.castka) AS obrat
FROM   Zakaznik z
JOIN   Objednavka o ON o.zakaznik_id = z.id
GROUP  BY z.id, z.jmeno, z.prijmeni
ORDER  BY obrat DESC
LIMIT  5;

-- 4) Zákazníci bez objednávky v 2025
SELECT z.id, z.jmeno, z.prijmeni
FROM   Zakaznik z
LEFT JOIN Objednavka o
        ON o.zakaznik_id = z.id AND YEAR(o.datum) = 2025
WHERE  o.id IS NULL;

-- 5) Produkty s prodejem > 100 ks
SELECT produkt, SUM(mnozstvi) AS kusu
FROM   Polozka
GROUP  BY produkt
HAVING SUM(mnozstvi) > 100
ORDER  BY kusu DESC;

-- 6) Statistika po městech
SELECT z.mesto,
       COUNT(DISTINCT z.id) AS pocet_zakazniku,
       AVG(o.castka)       AS prumer_objednavky
FROM   Zakaznik z
LEFT JOIN Objednavka o ON o.zakaznik_id = z.id
GROUP  BY z.mesto;

-- 7) Bonus: objednávky nad celkovým průměrem (CTE)
WITH avg_o AS (SELECT AVG(castka) AS prumer FROM Objednavka)
SELECT o.id, o.castka
FROM   Objednavka o, avg_o
WHERE  o.castka > avg_o.prumer
ORDER  BY o.castka DESC;

```

 Elevator pitch

REST API v ASP.NET Core implementujeme přes Controller (nebo Minimal API), který mapuje HTTP požadavky na metody. Framework se postará o routování, deserializaci JSON, validaci a vrací správné stavové kódy.

 TEORETICKÝ ZÁKLAD

- **REST** — zdroje (URL) + HTTP metody (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`) + JSON tělo + stavové kódy.
- **Controller-based API**: třída zděděná z `ControllerBase` s atributem `[ApiController]` a `[Route("api/[controller]")]`.
- **Routování**:
 - Atributové: `[HttpGet]`, `[HttpGet("{id:int})"]`, `[HttpPost]`.
 - Constraint `:int`, `:guid`, `:alpha`.
- **Model binding**:
 - `[FromRoute]` — z URL (`/api/products/42`).
 - `[FromQuery]` — z query stringu (`?page=2`).
 - `[FromBody]` — z těla (JSON).
 - `[FromHeader]`, `[FromForm]`.
- **Validate** přes data annotations: `[Required]`, `[StringLength(100)]`, `[Range(0, int.MaxValue)]`, `[EmailAddress]`. `[ApiController]` automaticky vrátí `400` při neplatném modelu.
- **DTO (Data Transfer Object)** — třída pro vstup/výstup API, oddělená od entit DB.
- **Návratové typy**: `Ok(data)` = 200, `Created(...)` = 201, `NoContent()` = 204, `NotFound()`, `BadRequest()`, `Unauthorized()`, `Conflict()`. Lze použít i `Results.*` v Minimal API.
- **DI (dependency injection)** — služby se registrují v `Program.cs`:
`builder.Services.AddScoped<IRepo, Repo>();` a injektují konstruktorem.
- **Swagger / OpenAPI** — automaticky generovaná dokumentace + UI
(`builder.Services.AddEndpointsApiExplorer(); AddSwaggerGen();`).
- **CORS** — povolení volání z jiného originu: `app.UseCors(p => p.WithOrigins("...").AllowAnyMethod());`.
- **Versioning**: `/api/v1/...`.
- **Asynchronní akce**: `public async Task<ActionResult<T>> Get() + await ...`.

 UKÁZKOVÉ ZADÁNÍ

Klíčová slova: ApiController, atributové routy, HTTP metody, DTO, ActionResult, validace, Swagger.

Zadání praktického úkolu

Vytvořte v ASP.NET Core REST API `BooksApi` pro správu knih. Datovou vrstvu zjednodušte na *in-memory* repozitář (`List<Book>`) — netřeba databáze.

1. Modely: třída `Book(id, title, author, year)` a DTO `BookCreateDto` (bez id).
2. Implementujte endpointy:
 - `GET /api/books` — seznam (volitelný query parametr `?author=`).
 - `GET /api/books/{id}` — detail; 404 pokud chybí.
 - `POST /api/books` — vytvoření; vrací 201 s `Location` hlavičkou.
 - `PUT /api/books/{id}` — nahrazení; 204 nebo 404.
 - `DELETE /api/books/{id}` — 204 nebo 404.
3. Validace DTO: `title` povinný (max 200 znaků), `year` mezi 0 a 2026.
4. Repozitář registrujte jako **singleton** v DI (`AddSingleton<IBookRepository, InMemoryBookRepository>()`).
5. Aktivujte Swagger UI a otestujte všechny endpointy.

? Otázky k obhajobě

- Rozdíl mezi POST a PUT?
- Proč používáme DTO oproti přímému použití entity?
- K čemu slouží `[ApiController]` ?

KUCHAŘKA — POSTUP

1. `dotnet new webapi -n BooksApi` .
2. Vytvořím `Models/Book.cs` a `Models/BookCreateDto.cs` s data annotations.
3. Rozhraní `IBookRepository` a třída `InMemoryBookRepository` v `Services/` .
4. Registrace v `Program.cs` : `AddSingleton` .
5. Controller `BooksController : ControllerBase` s atributy `[ApiController]` `[Route("api/[controller]")]` .
6. Implementuju 5 metod, vracím správné `ActionResult` .
7. `dotnet run` , otevřu `/swagger` a otestuji.

KLÍČOVÉ ÚRYVKY KÓDU

Models — entita a DTO s validací

```
using System.ComponentModel.DataAnnotations;

public class Book
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public string Author { get; set; } = "";
    public int Year { get; set; }
}

public class BookCreateDto
{
    [Required, StringLength(200)]
    public string Title { get; set; } = "";

    [Required, StringLength(100)]
    public string Author { get; set; } = "";

    [Range(0, 2026)]
    public int Year { get; set; }
}
```

BooksController.cs — celý CRUD

```

using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class BooksController : ControllerBase
{
    private readonly IBookRepository repo;
    public BooksController(IBookRepository repo) => this.repo = repo;

    [HttpGet]
    public ActionResult<IEnumerable<Book>> GetAll([FromQuery] string? author)
    {
        var data = repo.GetAll();
        if (!string.IsNullOrEmpty(author))
            data = data.Where(b => b.Author.Contains(author,
                StringComparison.OrdinalIgnoreCase));
        return Ok(data);
    }

    [HttpGet("{id:int}")]
    public ActionResult<Book> GetById(int id)
    {
        var b = repo.Find(id);
        return b is null ? NotFound() : Ok(b);
    }

    [HttpPost]
    public ActionResult<Book> Create([FromBody] BookCreateDto dto)
    {
        var book = new Book {
            Title = dto.Title, Author = dto.Author, Year = dto.Year
        };
        repo.Add(book);
        return CreatedAtAction(nameof(GetById), new { id = book.Id }, book);
    }

    [HttpPut("{id:int}")]
    public IActionResult Update(int id, [FromBody] BookCreateDto dto)
    {
        var b = repo.Find(id);
        if (b is null) return NotFound();
        b.Title = dto.Title; b.Author = dto.Author; b.Year = dto.Year;
        return NoContent();
    }

    [HttpDelete("{id:int}")]
    public IActionResult Delete(int id) =>

```

```
        repo.Delete(id) ? NoContent() : NotFound();  
    }  
}
```

Program.cs — registrace a Swagger

```
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddControllers();  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
builder.Services.AddSingleton<IBookRepository, InMemoryBookRepository>();  
  
var app = builder.Build();  
app.UseSwagger();  
app.UseSwaggerUI();  
app.MapControllers();  
app.Run();
```

18 Razor Pages — zpracování požadavku

Elevator pitch

Razor Pages je stránkově orientovaný model v ASP.NET Core. Každá stránka je dvojice `.cshtml` (HTML+Razor) + `.cshtml.cs` (PageModel s handlery). GET a POST se zpracovávají v metodách `OnGet` / `OnPost`.

TEORETICKÝ ZÁKLAD

- **Struktura stránky:** `Pages/Register.cshtml` + `Pages/Register.cshtml.cs` (PageModel třída).
- **Routování (file-based):** cesta v `Pages/` = URL. `Pages/Index.cshtml` = `/`,
`Pages/Produkty/Detail.cshtml` = `/Produkty/Detail`.
- **Pretty URI** — čisté URL bez koncovky `.aspx` nebo query stringu, např. `/Produkty/42` místo `/Produkty/Detail?id=42`. Dosahujeme přes route šablonu `@page "{id:int}"`.
- **Handlery PageModelu:**
 - `OnGet()` — při GET; připraví data pro zobrazení.
 - `OnPost()` — při POST; zpracuje formulář.
 - Pojmenované: `OnPostDelete()` volané přes `asp-page-handler="Delete"`.
 - Async varianty: `OnGetAsync()`, `OnPostAsync()`.
- **Bindování dat:**
 - `[BindProperty]` `public InputModel Input { get; set; }` — váže se na POST.
 - `[BindProperty(SupportsGet = true)]` — i z GET (query/route).
 - `ModelState.IsValid` — validace přes data annotations.
- **Návratové metody:**
 - `Page()` — znovu vykreslí stejnou stránku (typicky při neplatné validaci).
 - `RedirectToPage("Confirm")` — PRG (Post-Redirect-Get) vzor; zabrání duplicitnímu odeslání.
 - `RedirectToPage("Confirm", new { id = 5 })` — s parametry.
 - `NotFound()`, `Content("...")`, `File(...)`.
- **Předávání dat mezi stránkami:**
 - Route values: `RedirectToPage("Confirm", new { name, email })`.
 - `TempData["k"] = v;` — slovník přežívající jeden přesměrování (uloženo v session).
- **GET vs. POST:**
 - GET: data v URL, idempotentní, lze cachovat, pro *čtení*.
 - POST: data v těle, neidempotentní, pro *změnu stavu*.
- **Tag Helpery** v `.cshtml`: `asp-for`, `asp-page`, `asp-route-id`, `asp-page-handler`, `asp-validation-for`.

TÉMA Č. 18 — RAZOR PAGES

Klíčová slova: RazorPages, GET a POST požadavky, bindování dat, návratové metody (Redirect, View), routování, pretty URI.

Zadání praktického úkolu

Vytvořte jednoduchou webovou aplikaci v ASP.NET Core pomocí Razor Pages, která bude sloužit jako evidenční formulář pro registraci účastníka kurzu.

1. Na stránce `/Register` :

- Vytvořte formulář pro zadání: jméno, příjmení, e-mail, zvolený kurz (výběr z několika možností).
- Formulář zpracujte pomocí POST metody (`OnPost`) a při správném a úplném vyplnění výsledek přesměrujte na potvrzovací stránku.
- Umožněte naplnit formulářový prvek na základě parametru předaného stránce v URL (například předvyplnění e-mailu pomocí parametru `email`).

2. Na stránce `/Confirm` :

- Zobrazte zadané údaje z předchozího formuláře.
- Pro přenos dat mezi stránkami využijte slovník `TempData` nebo přesměrování s parametry (route values).

? Otázky k obhajobě

- Vysvětlete, jak v Razor Pages funguje routování a co znamená pojem *pretty URI*.
- Popište rozdíl mezi GET a POST požadavkem v kontextu zpracování formulářů.

 KUCHARKA — POSTUP

1. `dotnet new webapp -n CourseRegister` .
2. Vytvořím `Pages/Register.cshtml` + `.cshtml.cs` .
3. V PageModelu `RegisterModel` přidám vnořený `InputModel` s data annotations a označím `[BindProperty]` `public InputModel Input { get; set; }` .
4. `OnGet(string? email)` : pokud je email vyplněn, předvyplním `Input.Email = email;` .
5. `OnPost()` : zkontroluji `ModelState.IsValid` , při chybě vrátím `Page()` ; jinak uložím data do `TempData` a `RedirectToPage("Confirm")` .
6. Vytvořím `Pages/Confirm.cshtml` + `.cs` : v `OnGet()` vyzvednu hodnoty z `TempData` .
7. Otestuji v prohlížeči: `/Register?email=ja@x.cz` → vyplnění → submit → redirect na `/Confirm` .

KLÍČOVÉ ÚRYVKY KÓDU

Pages/Register.cshtml.cs — PageModel s handlery

```
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class RegisterModel : PageModel
{
    public class InputModel
    {
        [Required, StringLength(50)]
        public string Jmeno { get; set; } = "";

        [Required, StringLength(50)]
        public string Prijmeni { get; set; } = "";

        [Required, EmailAddress]
        public string Email { get; set; } = "";

        [Required]
        public string Kurz { get; set; } = "";
    }

    [BindProperty]
    public InputModel Input { get; set; } = new();

    public List<string> Kurzy { get; } =
        new() { "C# pro začátečníky", "Webové API", "React UI" };

    public void OnGet(string? email)
    {
        if (!string.IsNullOrEmpty(email)) Input.Email = email;
    }

    public IActionResult OnPost()
    {
        if (!ModelState.IsValid) return Page();

        TempData["Jmeno"]    = Input.Jmeno;
        TempData["Prijmeni"] = Input.Prijmeni;
        TempData["Email"]    = Input.Email;
        TempData["Kurz"]     = Input.Kurz;

        return RedirectToPage("Confirm");
    }
}
```

```
@page
@model RegisterModel
@{ ViewData["Title"] = "Registrace"; }

<h1>Registrace na kurz</h1>

<form method="post">
  <div>
    <label asp-for="Input.Jmeno"></label>
    <input asp-for="Input.Jmeno" />
    <span asp-validation-for="Input.Jmeno"></span>
  </div>
  <div>
    <label asp-for="Input.Prijmeni"></label>
    <input asp-for="Input.Prijmeni" />
    <span asp-validation-for="Input.Prijmeni"></span>
  </div>
  <div>
    <label asp-for="Input.Email"></label>
    <input asp-for="Input.Email" type="email" />
    <span asp-validation-for="Input.Email"></span>
  </div>
  <div>
    <label asp-for="Input.Kurz"></label>
    <select asp-for="Input.Kurz"
      asp-items="@((new SelectList(Model.Kurzy)))">
      <option value="">-- vyberte kurz --</option>
    </select>
    <span asp-validation-for="Input.Kurz"></span>
  </div>
  <button type="submit">Registrovat</button>
</form>
```



```
public class ConfirmModel : PageModel
{
    public string? Jmeno    { get; set; }
    public string? Prijmeni { get; set; }
    public string? Email    { get; set; }
    public string? Kurz     { get; set; }

    public void OnGet()
    {
        Jmeno    = TempData["Jmeno"]    as string;
        Prijmeni = TempData["Prijmeni"] as string;
        Email    = TempData["Email"]    as string;
        Kurz     = TempData["Kurz"]     as string;
    }
}
```

```
@page
@model ConfirmModel
<h1>Děkujeme za registraci!</h1>
<p>Účastník: @Model.Jmeno @Model.Prijmeni</p>
<p>Email: @Model.Email</p>
<p>Kurz: @Model.Kurz</p>
<a asp-page="/Register">Nová registrace</a>
```

19 ASP.NET Tag Helpers a formuláře

Elevator pitch

Tag Helpery jsou serverová rozšíření HTML značek v Razoru. Atributy `asp-*` zařídí generování správných URL, popisků, validací a vazbu na model. Nahrazují starší syntaxi `@Html.*` a vypadají jako čisté HTML.

TEORETICKÝ ZÁKLAD

- **Co jsou Tag Helpers** — třídy, které pluginují do parsingu Razoru a transformují tagy s `asp-*` atributy na výsledné HTML.
- **Aktivace** — `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers` v `_ViewImports.cshtml` (default v šabloně).
- **Klíčové atributy:**
 - `asp-for="Property"` — propojí `<input>` / `<label>` s vlastností modelu (název, id, type, validační atributy).
 - `asp-validation-for="Property"` — místo, kde se zobrazí chybová hláška.
 - `asp-validation-summary="ModelOnly|All"` — souhrn chyb.
 - `asp-items="..."` — naplní `<select>` kolekcí (typicky `SelectList`).
 - `asp-page` , `asp-page-handler` , `asp-controller` , `asp-action` , `asp-route-{název}` — routy a handlers (nahrazuje `href` / `action`).
 - `asp-antiforgery="true"` — automaticky token pro CSRF (default).
 - `asp-append-version="true"` — cache busting u `<link>` / `<script>` .
- **Vstupní prvky:** `<input>` , `<textarea>` , `<select>` , `<label>` , `<form>` — všechny mají TagHelper variantu.
- **Validace:**
 - Atributy modelu: `[Required]` , `[StringLength]` , `[Range]` , `[EmailAddress]` , `[RegularExpression]` , `[Compare]` .
 - Server: `ModelState.IsValid` .
 - Klient: jQuery Unobtrusive Validation (default v šabloně) — vykreslí `data-val-*` atributy a hlášky bez round-tripu.
- **Validace klient x server:** klient je rychlý a uživatelsky přívětivý, ale lze ho obejít — server musí vždy validovat.
- **Anti-forgery:** formuláře v Razoru automaticky generují skrytý token `__RequestVerificationToken` ; server jej ověřuje proti CSRF.
- **Vlastní Tag Helper** — třída : `TagHelper` s metodou `Process` ; aktivace přes `@addTagHelper` .

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: Tag Helpers (`asp-for`, `asp-items`, `asp-page-handler`, `asp-route-id`), validace klient/server, formuláře, handlers.

Výchozí stav

K dispozici máte rozpracovaný projekt v ASP.NET Core (Razor Pages). Vaším úkolem je aplikaci zprovoznit a zabezpečit doplněním chybějících Tag Helperů a validačních pravidel.

Zadání praktického úkolu

- Výběr kurzu (Index)** — Na titulní stránce se zobrazuje prvek `<select>` pro výběr kurzu. Po výběru kurzu je uživatel přesměrován na stránku pro přihlášení.
! Úkol: Chybí správné Tag Helpers (`asp-for` , `asp-items`) pro zobrazení seznamu kurzů. Doplňte je do příslušného souboru `.cshtml` .
- Přihlašovací formulář (Subscribe)** — Na této stránce se uživatel přihlásí pomocí jména a e-mailu. Formulář musí dále předávat i informaci o zvoleném kurzu. Úspěšné odeslání uloží přihlášku do databáze a přesměruje na seznam.
! Úkol: Je třeba doplnit `asp-*` atributy a validační atributy v `InputModel` , aby fungovalo ověření dat na straně serveru i klienta (např. povinná pole, správný formát e-mailu).
- Seznam přihlášek (List)** — Zobrazuje všechny odeslané přihlášky. U každé položky v seznamu jsou dvě tlačítka: tlačítko na zobrazení detailních údajů a tlačítko na smazání záznamu přes příslušný handler.
! Úkol: Doplňte Tag Helpers pro routování na handlers `Detail` a `Delete` (např. `asp-page-handler` , `asp-route-id`), aby tlačítka prováděla správné akce s konkrétním záznamem.

? Otázky k obhajobě

- Vysvětlete k čemu slouží Tag Helpers a jaký je jejich vztah k HTML značkám.
- Jaký je rozdíl mezi validací na straně klienta a na straně serveru? Proč jsou potřeba obě?

1. V `Index.cshtml` doplním `asp-for="SelectedKurzId"` a `asp-items="Model.KurzyList"` do `<select>`; v `PageModelu` naplním `KurzyList = new SelectList(...)`.
2. V `InputModel` přidám `[Required]`, `[EmailAddress]`, `[StringLength]`; v `Subscribe.cshtml` doplním `asp-for` a `asp-validation-for` a `<input type="hidden" asp-for="KurzId" />`.
3. V `Subscribe.cshtml.cs` v `OnPost`: `if (!ModelState.IsValid) return Page();`.
4. V `List.cshtml` u tlačítek doplním `asp-page-handler="Detail"` resp. `"Delete"` a `asp-route-id="@item.Id"`.
5. Přidám `OnPostDeleteAsync(int id)` a `OnPostDetail(int id)` (resp. GET handler) do `PageModelu`.
6. Pro klientskou validaci ověřím, že je v `_Layout` přilinkovaný `_ValidationScriptsPartial`.

KLÍČOVÉ ÚRYVKY KÓDU

`Index.cshtml` — výběr kurzu (`asp-items`)

```
@page
@model IndexModel

<form method="post">
  <label asp-for="SelectedKurzId">Kurz:</label>
  <select asp-for="SelectedKurzId"
    asp-items="Model.KurzyList">
    <option value="">-- vyberte --</option>
  </select>
  <button type="submit">Pokračovat</button>
</form>
```

```
public class IndexModel : PageModel
{
    private readonly AppDb db;
    public IndexModel(AppDb db) => this.db = db;

    [BindProperty] public int SelectedKurzId { get; set; }
    public SelectList KurzyList { get; private set; } = default!;

    public void OnGet()
    {
        KurzyList = new SelectList(db.Kurzy.ToList(), nameof(Kurz.Id), nameof(Ku

    }

    public IActionResult OnPost()
        => RedirectToPage("Subscribe", new { kurzId = SelectedKurzId });
}
```



 **Subscribe — InputModel s validací + skrytý KurzId**

```

public class SubscribeModel : PageModel
{
    public class InputModel
    {
        [Required(ErrorMessage = "Jméno je povinné")]
        [StringLength(80)]
        public string Jmeno { get; set; } = "";

        [Required, EmailAddress(ErrorMessage = "Neplatný e-mail")]
        public string Email { get; set; } = "";

        [Required] public int KurzId { get; set; }
    }

    [BindProperty] public InputModel Input { get; set; } = new();
    private readonly AppDb db;
    public SubscribeModel(AppDb db) => this.db = db;

    public void OnGet(int kurzId) => Input.KurzId = kurzId;

    public async Task<IActionResult> OnPostAsync()
    {
        if (!ModelState.IsValid) return Page();

        db.Prihlasky.Add(new Prihlaska {
            Jmeno = Input.Jmeno, Email = Input.Email, KurzId = Input.KurzId
        });
        await db.SaveChangesAsync();
        return RedirectToPage("List");
    }
}

```

```
@page "{kurzId:int}"
@model SubscribeModel

<form method="post">
  <input type="hidden" asp-for="Input.KurzId" />

  <label asp-for="Input.Jmeno"></label>
  <input asp-for="Input.Jmeno" />
  <span asp-validation-for="Input.Jmeno" class="text-danger"></span>

  <label asp-for="Input.Email"></label>
  <input asp-for="Input.Email" type="email" />
  <span asp-validation-for="Input.Email" class="text-danger"></span>

  <button type="submit">Přihlásit</button>
</form>

@section Scripts { <partial name="_ValidationScriptsPartial" /> }
```

List.cshtml — Detail/Delete přes handlery a route

```
@page
@model ListModel

<table>
  @foreach (var p in Model.Prihlasky)
  {
    <tr>
      <td>@p.Jmeno</td>
      <td>@p.Email</td>
      <td>
        <a asp-page="Detail" asp-route-id="@p.Id" class="btn btn-info">Detail</a>

        <form method="post" asp-page-handler="Delete"
              asp-route-id="@p.Id" style="display:inline">
          <button type="submit" class="btn btn-danger">Smazat</button>
        </form>
      </td>
    </tr>
  }
</table>
```



```
public class ListModel : PageModel
{
    private readonly AppDb db;
    public ListModel(AppDb db) => this.db = db;

    public List<Prihlaska> Prihlasky { get; set; } = new();

    public async Task OnGetAsync() =>
        Prihlasky = await db.Prihlasky.ToListAsync();

    public async Task<IActionResult> OnPostDeleteAsync(int id)
    {
        var p = await db.Prihlasky.FindAsync(id);
        if (p is null) return NotFound();
        db.Prihlasky.Remove(p);
        await db.SaveChangesAsync();
        return RedirectToPage();
    }
}
```


20 Next.js a serverové vs. klientské renderování

Elevator pitch

Next.js je React framework, který přidává file-based routing, server-side rendering, statické generování a optimalizace „out of the box“. Umožňuje pro každou stránku zvolit, kde a kdy se vykreslí — na serveru (rychlý first paint, SEO) nebo na klientovi (interaktivita).

TEORETICKÝ ZÁKLAD

- **Co Next.js přináší** oproti čistému Reactu: routing, SSR/SSG/ISR, image optimization, fonts, API endpointy, middleware, deploy na Vercel jedním kliknutím.
- **App Router (Next 13+)** — soubory ve složce `app/` :
 - `app/page.tsx` = `/` ,
 - `app/blog/page.tsx` = `/blog` ,
 - `app/blog/[slug]/page.tsx` = dynamický segment `/blog/:slug` ,
 - `app/layout.tsx` — společný layout pro podstrom.
- **Server Components (default)** — běží jen na serveru, nemají bundle u klienta, mohou volat databázi přímo. Žádné `useState` , `useEffect` .
- **Client Components** — vyžadují direktivu `"use client"` nahoře v souboru. Mají interaktivitu (state, events).
- **Druhy renderování:**
 - **SSG** (Static Site Generation) — HTML se vygeneruje při buildu (blog, dokumentace).
 - **SSR** (Server-Side Rendering) — HTML se vygeneruje při každém requestu (data v reálném čase, login).
 - **ISR** (Incremental Static Regeneration) — SSG, které se obnovuje na pozadí (e-shop kategorie). V App Routeru: `export const revalidate = 60` .
 - **CSR** (Client-Side Rendering) — komponenta se vykreslí v prohlížeči po načtení.
- **Načítání dat:** v Server Component prostě `await fetch(...)` ; v Client Component `useEffect` + `useState` nebo SWR/TanStack Query.
- **Cache fetchu:** default je cache; `fetch(url, { cache: 'no-store' })` = vždy fresh, `{ next: { revalidate: 60 } }` = ISR.
- **Server actions** — funkce s `"use server"` , lze volat přímo z formuláře jako `action={fn}` .
- **Routing primitivy:** `<Link href="/blog">` (klientská navigace bez full reloadu), `useRouter().push(...)` , `useParams()` , `useSearchParams()` .
- **API routes:** `app/api/products/route.ts` exportuje `GET` , `POST` handlers.
- **Loading a error stavy:** `loading.tsx` a `error.tsx` ve složce — automatické zobrazení.

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: Next.js App Router, Server vs. Client Component, SSR, SSG, ISR, CSR, fetch, dynamický segment.

Zadání praktického úkolu

Vytvořte v Next.js (App Router, TypeScript) malou aplikaci „TechBlog“ s následujícími stránkami:

1. `/` (Server Component, SSR) — seznam příspěvků načtený z veřejného API `https://jsonplaceholder.typicode.com/posts`. Každá položka je `<Link>` na detail.
2. `/posts/[id]` (Server Component, ISR) — detail příspěvku, `revalidate = 60` sekund. Pokud příspěvek neexistuje, vrátit `notFound()`.
3. `/search` (Client Component, CSR) — input a tlačítko, které načte články přes `useEffect` + `fetch` a klientsky filtruje podle obsahu.
4. Společný `app/layout.tsx` s navigačním pruhem a globálním fontem (next/font).
5. Vytvořte `app/loading.tsx` pro skeleton loader.

? Otázky k obhajobě

- Rozdíl mezi SSR a SSG?
- K čemu slouží direktiva `"use client"`?
- Jak Next.js cachuje fetch a jak to ovlivní?

KUCHARKA — POSTUP

1. `npx create-next-app@latest techblog --ts --app --eslint`.
2. Smažu obsah `app/page.tsx` a napíšu Server Component s `async function Page()`.
3. Vytvořím `app/posts/[id]/page.tsx`; používám `params.id`.
4. Pro CSR přidám `app/search/page.tsx` s prvním řádkem `"use client"`.
5. `app/loading.tsx` + `app/posts/[id]/error.tsx` pro automatické UI.
6. `npm run dev`; otestuji všechny stránky.

KLÍČOVÉ ÚRYVKY KÓDU

 `app/page.tsx` — Server Component (SSR fetch)

```

import Link from 'next/link';

type Post = { id: number; title: string; body: string };

export default async function HomePage() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts',
    { cache: 'no-store' }); // SSR – vždy fresh
  const posts: Post[] = await res.json();

  return (
    <main>
      <h1>TechBlog</h1>
      <ul>
        {posts.slice(0, 10).map(p => (
          <li key={p.id}>
            <Link href={`/${posts}/${p.id}`}>{p.title}</Link>
          </li>
        ))}
      </ul>
    </main>
  );
}

```

📄 app/posts/[id]/page.tsx — dynamický segment + ISR

```

import { notFound } from 'next/navigation';

export const revalidate = 60; // ISR: znovu vygeneruje po 60 s

export default async function PostPage(
  { params }: { params: { id: string } }
) {
  const res = await fetch(
    `https://jsonplaceholder.typicode.com/posts/${params.id}`,
    { next: { revalidate: 60 } }
  );
  if (!res.ok) notFound();
  const post = await res.json();

  return (
    <article>
      <h1>{post.title}</h1>
      <p>{post.body}</p>
    </article>
  );
}

```

```
'use client';
import { useEffect, useState } from 'react';

type Post = { id: number; title: string; body: string };

export default function SearchPage() {
  const [query, setQuery] = useState('');
  const [posts, setPosts] = useState<Post[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(r => r.json())
      .then((data: Post[]) => { setPosts(data); setLoading(false); });
  }, []);

  const filtered = posts.filter(p =>
    p.title.toLowerCase().includes(query.toLowerCase())
  );

  if (loading) return <p>Načítám...</p>;

  return (
    <main>
      <input value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Hľedať" />
      <ul>{filtered.map(p => <li key={p.id}>{p.title}</li>)}</ul>
    </main>
  );
}
```

 Elevator pitch

ORM (Object-Relational Mapping) přemostňuje objektový kód a relační databázi. Místo psaní SQL pracujeme s třídami a kolekcemi; ORM se postará o překlad. V .NET je standardem Entity Framework Core.

 TEORETICKÝ ZÁKLAD

- **Účel ORM:** méně boilerplate SQL, typová bezpečnost (kompilátor odhalí chybu), přenositelnost mezi DB systémy.
- **Klíčové ORM frameworky:** EF Core (.NET), Hibernate (Java), Prisma/TypeORM (Node.js), Django ORM (Python), SQLAlchemy.
- **Mapování:**
 - Třída ↔ tabulka.
 - Vlastnost ↔ sloupec.
 - Navigační vlastnost ↔ FK + JOIN.
- **EF Core hlavní pojmy:**
 - `DbContext` — brána k DB; obsahuje `DbSet<T>` pro každou entitu.
 - Konfigurace v `OnModelCreating` (Fluent API) nebo přes data annotations.
 - **Change tracking** — `DbContext` sleduje změny; `SaveChanges()` vygeneruje `INSERT/UPDATE/DELETE`.
- **Schéma a migrace:**
 - **Code First** — třídy → migrace → DB.
 - **Database First** — z existující DB se vygenerují třídy (`Scaffold-DbContext`).
 - Migrace: `dotnet ef migrations add Init` , `dotnet ef database update` .
- **Dotazování (LINQ to Entities):**
 - `db.Books.Where(b => b.Year >= 2020).OrderBy(b => b.Title).ToListAsync();` .
 - EF přeloží do SQL, vykoná na DB serveru.
 - `Include(b => b.Author)` — eager loading vazeb (řeší N+1 problém).
 - `AsNoTracking()` — pro read-only dotazy, lepší výkon.
- **Transakce** — implicitně kolem `SaveChanges` ; pro víc operací `BeginTransaction()` .
- **Rizika:** N+1 (zapomenutý `Include`), nadměrné načítání dat (žádný `Select` projekce), pomalé komplexní dotazy → ručně psané SQL nebo views.
- **Repository pattern** + ORM — abstrakce nad `DbContext` pro testovatelnost (často přehnané; EF samo má dost abstrakce).

 UKÁZKOVÉ ZADÁNÍ

Klíčová slova: Entity Framework Core, DbContext, DbSet, Code First migrace, LINQ, Include, asynchronní dotazy.

Zadání praktického úkolu

Vytvořte konzolovou aplikaci v C# (.NET 8) využívající **Entity Framework Core** s SQLite (zero-config). Aplikace bude evidovat blog:

1. Definujte entity `Blog` (Id, Nazev, Autor, Posts) a `Post` (Id, Titulek, Obsah, BlogId, Blog).
2. Vytvořte `BlogDbContext : DbContext` s `DbSet<Blog>` a `DbSet<Post>` a SQLite providerem.
3. Vytvořte počáteční migraci a aplikujte ji (`dotnet ef migrations add Init , database update`).
4. V `Main` :
 - vložte 2 blogy a do každého 3 příspěvky,
 - vypište všechny blogy s jejich příspěvky (`Include`),
 - najděte příspěvky obsahující slovo „React“,
 - aktualizujte titulek prvního příspěvku a uložte,
 - smažte vybraný příspěvek.

? Otázky k obhajobě

- Co je `DbContext` a co dělá `SaveChanges` ?
- Co je N+1 problém a jak ho řeší `Include` ?
- Rozdíl mezi Code First a Database First?

KUCHARKA — POSTUP

1. `dotnet new console -n EFBlog` ; přidám balíčky:
`Microsoft.EntityFrameworkCore.Sqlite , Microsoft.EntityFrameworkCore.Design` .
2. Definuji entity a vztahy přes navigation properties.
3. Vytvořím `BlogDbContext` s `OnConfiguring(o => o.UseSqlite("Data Source=blog.db"))` .
4. `dotnet ef migrations add Init , dotnet ef database update` .
5. V `Main` používám `using var db = new BlogDbContext();` a operace.
6. Asynchronní LINQ: `await db.Blogs.Include(b => b.Posts).ToListAsync();` .

KLÍČOVÉ ÚRYVKY KÓDU

Models + DbContext

```
using Microsoft.EntityFrameworkCore;

public class Blog
{
    public int Id { get; set; }
    public string Nazev { get; set; } = "";
    public string Autor { get; set; } = "";
    public List<Post> Posts { get; set; } = new();
}

public class Post
{
    public int Id { get; set; }
    public string Titulek { get; set; } = "";
    public string Obsah { get; set; } = "";
    public int BlogId { get; set; }
    public Blog? Blog { get; set; }
}

public class BlogDbContext : DbContext
{
    public DbSet<Blog> Blogs { get; set; }
    public DbSet<Post> Posts { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder o)
        => o.UseSqlite("Data Source=blog.db");
}
```

Program.cs — CRUD scénář

```

using var db = new BlogDbContext();
db.Database.EnsureCreated();

// Insert
if (!db.Blogs.Any())
{
    db.Blogs.AddRange(
        new Blog {
            Nazev = "Web", Autor = "Anna",
            Posts = new() {
                new Post { Titulek = "React 19", Obsah = "... " },
                new Post { Titulek = "Next.js", Obsah = "... " },
                new Post { Titulek = "TypeScript", Obsah = "... " }
            }
        },
        new Blog {
            Nazev = "Backend", Autor = "Petr",
            Posts = new() {
                new Post { Titulek = "ASP.NET 8", Obsah = "... " },
                new Post { Titulek = "EF Core", Obsah = "... " },
                new Post { Titulek = "PostgreSQL", Obsah = "... " }
            }
        }
    );
    await db.SaveChangesAsync();
}

// Eager loading - vyhne se N+1
var blogs = await db.Blogs
    .Include(b => b.Posts)
    .AsNoTracking()
    .ToListAsync();

foreach (var b in blogs)
{
    Console.WriteLine($"📁 {b.Nazev} ({b.Autor})");
    foreach (var p in b.Posts)
        Console.WriteLine($"    - {p.Titulek}");
}

// Filtr
var reactPosts = await db.Posts
    .Where(p => p.Titulek.Contains("React") || p.Obsah.Contains("React"))
    .ToListAsync();

// Update
var first = await db.Posts.FirstAsync();
first.Titulek = first.Titulek + " (aktualizováno)";

```



```
await db.SaveChangesAsync();
```

```
// Delete
```

```
var toDel = await db.Posts.OrderByDescending(p => p.Id).FirstAsync();
```

```
db.Posts.Remove(toDel);
```

```
await db.SaveChangesAsync();
```

22 React — komponenty a props

Elevator pitch

Komponenta je samostatná, znovupoužitelná část UI. V moderním Reactu se píše jako funkce, která vrací JSX. Props jsou neměnné vstupy z rodiče — komponenta podle nich rozhoduje, co vykreslit.

TEORETICKÝ ZÁKLAD

- **Funkcionální komponenta:**

```
function Hello({ name }: { name: string }) {  
  return <h1>Ahoj {name}</h1>;  
}
```

- **JSX:** rozšíření JS, transpiluje se na `React.createElement` ; výrazy v `{ }` ; atributy v camelCase (`className` , `onClick`).
- **Props (properties):**
 - Jednosměrný tok shora dolů (parent → child).
 - Read-only — komponenta props nemění.
 - Destrukturujeme v signatuře: `function Card({ title, onClick })` .
 - TypeScript: deklarujeme typ propů (`type Props = { title: string; }`).
 - Default hodnoty: `function Card({ title = "Bez názvu" })` .
- **Speciální prop `children`** — obsah mezi otevíracím a zavíracím tagem komponenty.
- **Předávání callback funkci:** rodič předá funkci, dítě ji zavolá při události — tak putuje informace zezdola nahoru.
- **Klíče v seznámeních:** `map` generuje JSX → každý prvek potřebuje stabilní `key` (typicky id z dat, nikdy index, pokud se pořadí mění).
- **Skládání (composition)** — komponenty obalujeme do jiných (`<Card><Avatar /></Card>`).
- **Pojmenování:** komponenty velkým písmenem (`MyButton`); HTML tagy malým.
- **Re-render** nastane, když se změní props nebo vlastní state. Kontrola: `React.memo` , `useMemo` , `useCallback` .
- **Prop drilling** — propsy se proplétají skrz mnoho úrovní; řeší to `Context` nebo state-management.

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: funkcionální komponenta, JSX, props, children, callback funkce, mapování seznamu, klíče.

Zadání praktického úkolu

Vytvořte v Reactu (Vite + TypeScript) jednoduchou aplikaci „Katalog produktů“:

1. Pole produktů (10 položek) je v `App.tsx` : `{id, nazev, cena, novinka}` .
2. Komponenta `<ProductCard product onAddToCart />` zobrazí název, cenu a tlačítko „Do košíku“. Pokud `novinka` je `true`, zobrazí badge.
3. Komponenta `<ProductList products onAddToCart />` dostává pole a callback, mapuje na `ProductCard` .
4. Komponenta `<Header>` přijímá `children` (slogan) a počet položek v košíku jako prop.
5. V `App` : state s počtem položek košíku; po kliknutí na tlačítko se inkrementuje a v Headeru se okamžitě zobrazí.
6. Použijte TypeScript a definujte typ `Product` .

? Otázky k obhajobě

- Co jsou props a proč jsou read-only?
- Jak putuje informace z dítěte do rodiče?
- K čemu slouží `key` v seznamu?

KUCHARKA — POSTUP

1. `npm create vite@latest katalog -- --template react-ts` .
2. Vytvořím soubor `types.ts` s `export type Product = {...}` .
3. Komponenty `ProductCard.tsx` , `ProductList.tsx` , `Header.tsx` v `src/components/` .
4. V `App.tsx` držím `const [count, setCount] = useState(0)` .
5. Funkce `handleAdd = (id) => setCount(c => c + 1)` předám jako prop.
6. Otestuji v devtools React Components panelu.

KLÍČOVÉ ÚRYVKY KÓDU

 `types.ts` + komponenty

```
// types.ts
export type Product = {
  id: number;
  nazev: string;
  cena: number;
  novinka?: boolean;
};

// ProductCard.tsx
import type { Product } from '../types';

type Props = {
  product: Product;
  onAddToCart: (id: number) => void;
};

export default function ProductCard({ product, onAddToCart }: Props) {
  return (
    <article className="card">
      {product.novinka && <span className="badge">NOVINKA</span>}
      <h3>{product.nazev}</h3>
      <p>{product.cena.toFixed(2)} Kč</p>
      <button onClick={() => onAddToCart(product.id)}>
        Do košíku
      </button>
    </article>
  );
}
```

```
// ProductList.tsx
import ProductCard from './ProductCard';
import type { Product } from '../types';

type Props = {
  products: Product[];
  onAddToCart: (id: number) => void;
};

export default function ProductList({ products, onAddToCart }: Props) {
  return (
    <div className="list">
      {products.map(p => (
        <ProductCard
          key={p.id}                {/* stabilní klíč */}
          product={p}
          onAddToCart={onAddToCart}
        />
      ))}
    </div>
  );
}
```

```
// Header.tsx – children + prop
type Props = { children: React.ReactNode; cartCount: number };

export default function Header({ children, cartCount }: Props) {
  return (
    <header className="header">
      <h1>Katalog</h1>
      <p>{children}</p>
      <span>🛒 {cartCount}</span>
    </header>
  );
}
```

```
// App.tsx – propojení
import { useState } from 'react';
import Header from './components/Header';
import ProductList from './components/ProductList';
import type { Product } from './types';

const data: Product[] = [
  { id: 1, nazev: 'Káva', cena: 75, novinka: true },
  { id: 2, nazev: 'Croissant', cena: 45 },
  /* ... */
];

export default function App() {
  const [count, setCount] = useState(0);
  const handleAdd = (_id: number) => setCount(c => c + 1);

  return (
    <
      <Header cartCount={count}>
        Nejlepší kavárna ve městě
      </Header>
      <ProductList products={data} onAddToCart={handleAdd} />
    </>
  );
}
```

23 React — hooks (useState, useEffect aj.)

Elevator pitch

Hooks jsou funkce, které „připojují“ funkcionální komponenty na stav, životní cyklus a další schopnosti Reactu. Místo třídních `this.state` a `componentDidMount` dnes používáme `useState`, `useEffect`, `useRef` a další.

TEORETICKÝ ZÁKLAD

- **Pravidla hooks:**
 - Volat jen na nejvyšší úrovni komponenty (ne v podmínce, smyčce ani po `return`).
 - Volat pouze v Reactových komponentách nebo v dalších custom hookech.
- **useState** — drží lokální stav komponenty: `const [count, setCount] = useState(0);`. Setter může přijmout funkci: `setCount(c => c + 1)` — vždy aktuální hodnota.
- **useEffect** — vedlejší efekty (fetch, timer, subscriptions):
 - `useEffect(() => { ... }, [])` — po prvním renderu (~`componentDidMount`).
 - `useEffect(() => { ... }, [dep])` — při změně závislosti.
 - Bez deps — po každém renderu (zřídka).
 - **Cleanup:** `return () => clearInterval(id);` — uklidí předchozí efekt nebo při `odmount`.
- **useRef** — neměnný kontejner přes rendery:
 - Reference na DOM uzel: `<input ref={inputRef} />`; `inputRef.current.focus();`.
 - Uchování hodnoty bez re-renderu (např. ID intervalu).
- **useId** — generuje unikátní ID pro `labely/aria` atributy (stabilní mezi serverem a klientem).
- **useMemo** — cache výsledku drahého výpočtu, dokud se nezmění závislosti.
- **useCallback** — memoizovaná verze funkce, aby se neměnila reference (užitečné s `React.memo` a `useEffect` deps).
- **useContext** — čte hodnotu z `Context` u (téma, jazyk, auth).
- **useReducer** — alternativa k `useState` pro složitější stav (akce + reducer).
- **Custom hook** — funkce začínající `use`, která zapouzdřuje logiku (`useFetch`, `useDebounce`).
- **Strict mode** v dev režimu spouští efekty 2× — odhalí špatný cleanup.

UKÁZKOVÉ ZADÁNÍ

Klíčová slova: `useState`, `useEffect`, `useRef`, `useId`, custom hook, `debounce`, `cleanup`.

Zadání praktického úkolu

Vytvořte v Reactu (Vite + TS) stránku „Vyhledávač uživatelů“ s následujícími požadavky:

1. Vstupní pole pro hledání jména. Po prvním renderu se mu nastaví focus přes `useRef`.
2. Když uživatel přestane psát na 400 ms, aplikace zavolá `https://jsonplaceholder.typicode.com/users?q=...` (klientsky filtruje, `debounce`). Implementujte vlastní hook `useDebounce(value, delay)`.
3. Stav: `loading`, `data`, `error` (3 `useState`).
4. Po odmountu vyhledávání zrušit (`AbortController`).
5. Pro každý výsledek vygenerujte unikátní `id` přes `useId` a propojte `label` ↔ `checkbox` v seznamu.
6. Tlačítko „Reset“ vyčistí výsledky a vrátí focus do inputu.

? Otázky k obhajobě

- Proč `useEffect` vrací `cleanup` funkci?
- Co je `debounce` a kdy ho použít?
- Rozdíl mezi `useRef` a `useState`?

KUCHARKA — POSTUP

1. Vytvořím `useDebounce.ts` — uvnitř `useState` + `useEffect` s `setTimeout` a `cleanup`em `clearTimeout`.
2. V hlavní komponentě: `const debounced = useDebounce(query, 400);`
3. `useEffect(() => { fetch(...), [debounced] })` — `fetch` jen když se změní `debounced` hodnota.
4. `AbortController`: vytvořím novou v efektu, předám do `fetch(..., { signal })`, v `cleanup` zavolám `abort()`.
5. `inputRef = useRef<HTMLInputElement>(null);` v `useEffect(() => inputRef.current?.focus(), [])`.
6. `const baseId = useId();` a v map klíč `const cbId = `${baseId}-${user.id}``.

KLÍČOVÉ ÚRYVKY KÓDU

 `useDebounce` — vlastní hook


```
import { useEffect, useState } from 'react';

export function useDebounce<T>(value: T, delay = 400): T {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);    // cleanup → zruší starý timer
  }, [value, delay]);

  return debounced;
}
```

 UserSearch.tsx — useState/useEffect/useRef/useId/AbortController

```

import { useEffect, useId, useRef, useState } from 'react';
import { useDebounce } from './useDebounce';

type User = { id: number; name: string; email: string };

export default function UserSearch() {
  const inputRef = useRef<HTMLInputElement>(null);
  const baseId = useId();

  const [query, setQuery] = useState('');
  const debounced = useDebounce(query, 400);
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  // Auto-focus po mountu
  useEffect(() => { inputRef.current?.focus(); }, []);

  // Fetch při změně debouncované hodnoty
  useEffect(() => {
    if (debounced.trim() === '') { setUsers([]); return; }

    const ctrl = new AbortController();
    setLoading(true);
    setError(null);

    fetch(`https://jsonplaceholder.typicode.com/users?q=${debounced}`,
      { signal: ctrl.signal })
      .then(r => r.json())
      .then((data: User[]) => setUsers(data))
      .catch(e => { if (e.name !== 'AbortError') setError(e.message); })
      .finally(() => setLoading(false));

    return () => ctrl.abort(); // cleanup
  }, [debounced]);

  return (
    <section>
      <input ref={inputRef}
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Hledat uživatele..." />

      <button onClick={() => {
        setQuery(''); setUsers([]); inputRef.current?.focus();
      }}>Reset</button>
    </section>
  );
}

```

```
{loading && <p>Načítám...</p>}
{error && <p style={{color:'red'}}>{error}</p>}

<ul>
  {users.map(u => {
    const cbId = `${baseId}-${u.id}`;
    return (
      <li key={u.id}>
        <input id={cbId} type="checkbox" />
        <label htmlFor={cbId}>{u.name} ({u.email})</label>
      </li>
    );
  })}
</ul>
</section>
);
}
```

💡 Elevator pitch

React Router je standardní routovací knihovna pro klientské React SPA — propojuje URL s komponentami, takže aplikace se chová jako klasický web (zpět/vpřed, sdílení odkazů), ale bez znovunačtení stránky.

📺 TEORETICKÝ ZÁKLAD

- **Instalace:** `npm i react-router-dom`.
- **Hlavní stavební kameny:**
 - `<BrowserRouter>` — kořenový provider routeru (založený na History API). V hl. souboru obalí `<App />`.
 - `<Routes>` — kontejner; vybere *jednu* shodující se rotu.
 - `<Route path="/users/:id" element={<UserDetail />} />` — definice routy.
 - `<Link to="/about">` — náhrada `<a>`, navigace bez full reloadu.
 - `<NavLink>` — `Link`, který přidá `active` třídu, když je cesta aktivní.
 - `<Navigate to="/login" replace />` — programatické přesměrování.
 - `<Outlet />` — místo, kde se vykreslí potomek nested routy.
- **Hooks:**
 - `useNavigate()` — funkce pro programové přesměrování (`navigate('/home')` , `navigate(-1)`).
 - `useParams()` — parametry z URL (`{ id }`).
 - `useSearchParams()` — query string.
 - `useLocation()` — aktuální path/state.
- **Nested routes** — vnořené cesty:

```
<Route path="/admin" element={<AdminLayout />}>
  <Route index element={<Dashboard />} />
  <Route path="users" element={<Users />} />
</Route>
```
- **Dynamické segmenty:** `path="users/:id"` ; v komponentě `const { id } = useParams();` .
- **404 / fallback:** `<Route path="*" element={<NotFound />} />` .
- **Chráněné routy:** wrapper komponenta, která zkontroluje login a buď vrátí `<Outlet />` , nebo `<Navigate to="/login" />` .
- **Data Router (v6.4+)** — `createBrowserRouter` + `RouterProvider` umožňuje `loader` , `action` , `errorElement` přímo v deklaraci rout.

- **Rozdíl Link vs. <a>** : `<a>` reloaduje celou aplikaci, `Link` jen překreslí odpovídající komponentu.

UKÁZKOVÉ ZADÁNÍ

TÉMA Č. 24 — REACT ROUTER

Klíčová slova: `BrowserRouter`, `Routes`, `Route`, `Link`, `NavLink`, `useParams`, `useNavigate`, nested routes, chráněná ruta.

Zadání praktického úkolu

Vytvořte v Reactu (Vite + TS) SPA s React Routerem:

1. **Layout** s navigací (`NavLink`) na: Domů, Produkty, O nás, Login.
2. **Trasy:**
 - `/` — uvítací stránka,
 - `/produkty` — seznam (statické pole), každý produkt je `<Link to={}`/produkty/${id}`>` ,
 - `/produkty/:id` — detail; pokud id neexistuje, `useNavigate` přesměruje na `/404` ,
 - `/o-nas` — statická stránka,
 - `/login` — formulář (jen UI), po „přihlášení“ zapne flag v `localStorage` ,
 - `/admin` — chráněná ruta (`RequireAuth`), s nested routes `/admin` (přehled) a `/admin/users` ,
 - `*` — 404 stránka.
3. Zvýrazněte aktivní položku v navigaci přes `NavLink` + CSS.
4. Tlačítko *Zpět* na detailu produktu používá `navigate(-1)` .

? Otázky k obhajobě

- Rozdíl mezi `<Link>` a `<a>` ?
- Jak udělat chráněnou routu?
- K čemu slouží `<Outlet />` ?

KUCHARKA — POSTUP

1. `npm create vite@latest spa -- --template react-ts ; npm i react-router-dom .`
2. V `main.tsx` obalím `<App />` do `<BrowserRouter>` .
3. V `App.tsx` definuji `<Routes>` se všemi trasami; layout obsahuje `<Outlet />` .
4. Detail produktu používá `useParams` , najde položku, jinak `navigate('/404', {replace:true})` .
5. Komponenta `<RequireAuth>` kontroluje `localStorage` a vrací `<Outlet />` nebo `<Navigate to="/login" />` .
6. Otestuji navigaci, refresh, zpět/vpřed.

KLÍČOVÉ ÚRYVKY KÓDU

main.tsx — BrowserRouter

```
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

App.tsx — Routes + nested + chráněná

```

import { Routes, Route, Navigate, NavLink, Outlet } from 'react-router-dom';
import Home from './pages/Home';
import Products from './pages/Products';
import ProductDetail from './pages/ProductDetail';
import About from './pages/About';
import Login from './pages/Login';
import AdminDashboard from './pages/AdminDashboard';
import AdminUsers from './pages/AdminUsers';
import NotFound from './pages/NotFound';

```

```

function Layout() {
  return (
    <
      <nav>
        <NavLink to="/" end>Domů</NavLink>
        <NavLink to="/produkty">Produkty</NavLink>
        <NavLink to="/o-nas">O nás</NavLink>
        <NavLink to="/admin">Admin</NavLink>
        <NavLink to="/login">Login</NavLink>
      </nav>
      <main><Outlet /></main>
    </>
  );
}

```

```

function RequireAuth() {
  const isLoggedIn = localStorage.getItem('logged') === '1';
  return isLoggedIn ? <Outlet /> : <Navigate to="/login" replace />;
}

```

```

export default function App() {
  return (
    <Routes>
      <Route element={<Layout />}>
        <Route path="/" element={<Home />} />
        <Route path="produkty" element={<Products />} />
        <Route path="produkty/:id" element={<ProductDetail />} />
        <Route path="o-nas" element={<About />} />
        <Route path="login" element={<Login />} />

        <Route element={<RequireAuth />}>
          <Route path="admin" element={<AdminDashboard />} />
          <Route path="admin/users" element={<AdminUsers />} />
        </Route>

        <Route path="*" element={<NotFound />} />
      </Route>
    </Routes>
  );
}

```

```
    </Routes>
  );
}
```

ProductDetail.tsx — useParams + useNavigate

```
import { useParams, useNavigate } from 'react-router-dom';
import { products } from '../data';

export default function ProductDetail() {
  const { id } = useParams();
  const navigate = useNavigate();
  const product = products.find(p => p.id === Number(id));

  if (!product) {
    navigate('/404', { replace: true });
    return null;
  }

  return (
    <article>
      <h1>{product.name}</h1>
      <p>{product.price} Kč</p>
      <button onClick={() => navigate(-1)}>Zpět</button>
    </article>
  );
}
```


25 Správa stavů aplikace v Reactu

💡 Elevator pitch

Stav je informace, kterou si komponenta pamatuje mezi rendery. Malý lokální stav řeší `useState`, sdílený mezi sourozenci „lifting state up“, napříč aplikací `Context`, složitější přechody `useReducer`. Pro velké projekty existují knihovny (Zustand, Redux Toolkit, TanStack Query).

📺 TEORETICKÝ ZÁKLAD

• Druhy stavu:

- **Lokální UI** — otevřený modal, vstup formuláře (`useState`).
- **Sdílený mezi komponentami** — košík, přihlášený uživatel, téma (Context, store).
- **Server state / data cache** — výsledky API (TanStack Query, SWR — automatický caching, refetch).
- **URL state** — co je v URL (`useSearchParams`).

• **Lifting state up** — když dvě komponenty potřebují tutéž informaci, přesuneme stav do nejbližšího společného předka a předáváme přes props.

• **Context API** — vyhne se prop drillingu:

- `const ThemeContext = createContext<Theme>('light');`
- `<ThemeContext.Provider value={...}>...</Provider>`
- `const theme = useContext(ThemeContext);`
- Pozor: změna value přerenderuje všechny consumenty → vhodné pro málo se měnící data (téma, jazyk, auth).

• **useReducer** — pro stav s několika přechody:

```
type Action = { type: 'add'; item: Item }
              | { type: 'remove'; id: number }
              | { type: 'clear' };

function reducer(state: Item[], a: Action): Item[] {
  switch (a.type) {
    case 'add':    return [...state, a.item];
    case 'remove': return state.filter(i => i.id !== a.id);
    case 'clear':  return [];
  }
}

const [items, dispatch] = useReducer(reducer, []);
dispatch({ type: 'add', item });
```

• **Context + useReducer** — chudákova alternativa Reduxu, plně postačí pro většinu aplikací.

- **Knihovny:**
 - **Zustand** — minimalistický store, hook-based.
 - **Redux Toolkit** — průmyslový standard pro velké aplikace; slices, immer, devtools.
 - **TanStack Query (React Query)** — server state (cache, retry, refetch on focus).
- **Imutabilita** — stav nikdy nemutujeme přímo (`state.push(x)` ❌); vytváříme nový (`[...state, x]` ✅). Reducer musí být čistá funkce.
- **Optimalizace re-renderů:** `memo` , `useMemo` , `useCallback` ; rozdělení Contextů (oddělit „často se mění“ od „skoro se nemění“).

UKÁZKOVÉ ZADÁNÍ

TÉMA Č. 25 — SPRÁVA STAVŮ V REACTU

Klíčová slova: `useState`, lifting state up, Context, `useReducer`, custom hook, imutabilita.

Zadání praktického úkolu

Vytvořte v Reactu (Vite + TS) aplikaci „Košík“ s globálně dostupným stavem košíku přes Context + `useReducer`:

1. Definujte typ `CartItem = { id, nazev, cena, mnozstvi }`.
2. Vytvořte `CartContext` s tvarem `{ items, addItem, removeItem, clear, total }` a `CartProvider`, který používá `useReducer`.
3. Akce reduceru: `ADD` (pokud existuje, zvýší množství), `REMOVE`, `CLEAR`.
4. Custom hook `useCart()` pro čistší použití (`const { items, addItem } = useCart();`).
5. Vytvořte komponenty: `<ProductList />` (volá `addItem`), `<CartIcon />` (zobrazuje počet), `<CartPage />` (seznam, total, tlačítko clear).
6. Persistuj stav do `localStorage` (load při mountu Provideru, save při změně).

? Otázky k obhajobě

- Kdy zvolit Context a kdy Redux/Zustand?
- Proč nesmíme stav mutovat přímo?
- Jak řeší `useReducer` složitý stav lépe než více `useState` ?

KUCHARKA — POSTUP

1. Definuji typy `CartItem` , `Action` , `State` .
2. Napíšu čistý reducer `cartReducer` bez side-effectů.
3. `CartContext.tsx` : `createContext` , `CartProvider` s `useReducer` , `useEffect` pro persistence.
4. Hook `useCart()` = `useContext(CartContext)` + kontrola na undefined (musí být v Provideru).
5. V `main.tsx` obalím app do `<CartProvider>` .
6. Komponenty volají `useCart()` — žádný prop drilling.

KLÍČOVÉ ÚRYVKY KÓDU

 `CartContext.tsx` — Context + `useReducer` + persistence

```

import {
  createContext, useContext, useEffect, useReducer, ReactNode
} from 'react';

export type CartItem = {
  id: number; nazev: string; cena: number; mnozstvi: number;
};

type State = CartItem[];
type Action =
  | { type: 'ADD'; item: Omit<CartItem, 'mnozstvi'> }
  | { type: 'REMOVE'; id: number }
  | { type: 'CLEAR' }
  | { type: 'INIT'; items: State };

function reducer(state: State, a: Action): State {
  switch (a.type) {
    case 'INIT': return a.items;
    case 'CLEAR': return [];
    case 'REMOVE': return state.filter(i => i.id !== a.id);
    case 'ADD': {
      const existing = state.find(i => i.id === a.item.id);
      if (existing) {
        return state.map(i =>
          i.id === a.item.id ? { ...i, mnozstvi: i.mnozstvi + 1 } : i
        );
      }
      return [...state, { ...a.item, mnozstvi: 1 }];
    }
  }
}

type CartCtx = {
  items: CartItem[];
  addItem: (i: Omit<CartItem, 'mnozstvi'>) => void;
  removeItem: (id: number) => void;
  clear: () => void;
  total: number;
};

const CartContext = createContext<CartCtx | undefined>(undefined);

export function CartProvider({ children }: { children: ReactNode }) {
  const [items, dispatch] = useReducer(reducer, []);

  // Load
  useEffect(() => {

```

```

    const raw = localStorage.getItem('cart');
    if (raw) dispatch({ type: 'INIT', items: JSON.parse(raw) });
  }, []);

  // Save
  useEffect(() => {
    localStorage.setItem('cart', JSON.stringify(items));
  }, [items]);

  const total = items.reduce((s, i) => s + i.cena * i.mnozstvi, 0);

  const value: CartCtx = {
    items,
    addItem: item => dispatch({ type: 'ADD', item }),
    removeItem: id => dispatch({ type: 'REMOVE', id }),
    clear: () => dispatch({ type: 'CLEAR' }),
    total,
  };

  return <CartContext.Provider value={value}>{children}</CartContext.Provider>;
}

export function useCart(): CartCtx {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error('useCart musí být uvnitř CartProvider');
  return ctx;
}

```

Použití v komponentách

```

// CartIcon.tsx
import { useCart } from './CartContext';
export default function CartIcon() {
  const { items } = useCart();
  const count = items.reduce((s, i) => s + i.mnozstvi, 0);
  return <span>🛒 {count}</span>;
}

// ProductList.tsx
import { useCart } from './CartContext';
export default function ProductList({ products }: { products: any[] }) {
  const { addItem } = useCart();
  return (
    <ul>
      {products.map(p => (
        <li key={p.id}>
          {p.nazev} – {p.cena} Kč
          <button onClick={() => addItem({ id: p.id, nazev: p.nazev, cena: p.cena })}>
            Přidat
          </button>
        </li>
      ))}
    </ul>
  );
}

// CartPage.tsx
import { useCart } from './CartContext';
export default function CartPage() {
  const { items, removeItem, clear, total } = useCart();
  if (items.length === 0) return <p>Košík je prázdný</p>;
  return (
    <div>
      <ul>
        {items.map(i => (
          <li key={i.id}>
            {i.nazev} × {i.mnozstvi} = {i.cena * i.mnozstvi} Kč
            <button onClick={() => removeItem(i.id)}>✕</button>
          </li>
        ))}
      </ul>
      <p><strong>Celkem: {total} Kč</strong></p>
      <button onClick={clear}>Vyprázdnit</button>
    </div>
  );
}

```

```
// main.tsx
// <CartProvider><App /></CartProvider>
```